

# NETWORK ANOMALY DETECTION

## SECTION 01: PROBLEM STATEMENT, DATASET & SETUP

### Network Anomaly Detection — Context

A **Network Intrusion Detection System (IDS)** monitors network traffic for malicious activity. Two broad approaches exist:

- **Signature-based:** matches traffic against a database of known attack patterns. Fast and precise, but blind to new (zero-day) attacks.
- **Anomaly-based:** learns a baseline of "normal" traffic (using ML) and flags deviations from it. Can catch unknown attacks, at the cost of more false positives.

This project builds an **anomaly-based** detector based on the ``Network_anomaly_data.csv`` data

### PROBLEM STATEMENT

In the realm of cybersecurity, network anomaly detection is a critical task that involves identifying unusual patterns or behaviors that deviate from the norm within network traffic. These anomalies could signify a range of security threats, from compromised devices and malware infections to large-scale cyber-attacks like DDoS (Distributed Denial of Service).

As a Data Scientist in the Cyber Security department, I am provided with networking data to visualise and analyse. My task is to perform exploratory data analysis and identify meaningful patterns and insights from the dataset.

I will apply Supervised Learning algorithms to identify the best-performing model for classifying network traffic using the attack column as the target variable. I will also apply Unsupervised Learning algorithms to identify the best model for detecting anomalies in the dataset without using the attack column.

Finally, I will deploy the selected Machine Learning models through a Flask API, enabling the models to be used for real-time classification and anomaly detection.

This project divided into four blocks -

1. Tableau Visualisations
2. EDA and Hypothesis Testing
3. ML Modeling
4. Deployment

## Attack categories

The 22 attack types in the raw attack column collapse into 4 categories + Normal:

```
In [ ]: attack_categories_dict = {
        "Normal": ['normal'],
        "DOS": ['back', 'land', 'neptune', 'pod', 'smurf', 'teardrop'],
        "R2L": ['ftp_write', 'guess_passwd', 'imap', 'multihop', 'phf', 'spy', 'ware'],
        "U2R": ['buffer_overflow', 'loadmodule', 'perl', 'rootkit'],
        "Probe": ['ipsweep', 'nmap', 'portsweep', 'satan']
    }
```

## Import libraries and load dataset

```
In [ ]: !pip install optuna
```

```
Collecting optuna
  Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)
Collecting alembic>=1.5.0 (from optuna)
  Downloading alembic-1.19.1-py3-none-any.whl.metadata (7.3 kB)
Collecting colorlog (from optuna)
  Downloading colorlog-6.12.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (26.2)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.51)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from optuna) (4.67.3)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from optuna) (6.0.3)
Requirement already satisfied: Mako in /usr/lib/python3/dist-packages (from alembic>=1.5.0->optuna) (1.1.3)
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (4.16.0)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.5.3)
Downloading optuna-4.9.0-py3-none-any.whl (425 kB)
_____ 425.6/425.6 kB 8.0 MB/s eta 0:00:00
Downloading alembic-1.19.1-py3-none-any.whl (265 kB)
_____ 265.9/265.9 kB 17.4 MB/s eta 0:00:00
Downloading colorlog-6.12.0-py3-none-any.whl (12 kB)
Installing collected packages: colorlog, alembic, optuna
Successfully installed alembic-1.19.1 colorlog-6.12.0 optuna-4.9.0
```

```
In [ ]: import warnings, math, pickle, time, os, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations, permutations

from scipy import stats
from scipy.stats import (ttest_ind, levene, anderson, mannwhitneyu,
                        f_oneway, kruskal, chi2_contingency)
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, AdaBoostClassifier,
                             GradientBoostingClassifier, IsolationForest)
from sklearn.covariance import EllipticEnvelope
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_s
                             classification_report, confusion_matrix, roc_auc_s
from imblearn.over_sampling import SMOTE

from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
import optuna
from optuna.samplers import TPESampler

warnings.filterwarnings("ignore")
optuna.logging.set_verbosity(optuna.logging.WARNING)
pd.set_option("display.max_columns", None)
RANDOM_STATE = 42

def display_all(df):
    with pd.option_context("display.max_rows", 100, "display.max_columns", 100):
        display(df)
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

## Getting the data

```
In [ ]: DATA_PATH = "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network And

nadp = pd.read_csv(DATA_PATH)
print(f"Rows: {nadp.shape[0]:,} | Columns: {nadp.shape[1]}")
nadp.head()
```

Rows: 125,973 | Columns: 43

Out[ ]:

|   | duration | protocoltype | service  | flag | srcbytes | dstbytes | land | wrongfragment | urg |
|---|----------|--------------|----------|------|----------|----------|------|---------------|-----|
| 0 | 0        | tcp          | ftp_data | SF   | 491      | 0        | 0    | 0             |     |
| 1 | 0        | udp          | other    | SF   | 146      | 0        | 0    | 0             |     |
| 2 | 0        | tcp          | private  | S0   | 0        | 0        | 0    | 0             |     |
| 3 | 0        | tcp          | http     | SF   | 232      | 8153     | 0    | 0             |     |
| 4 | 0        | tcp          | http     | SF   | 199      | 420      | 0    | 0             |     |



## Dataset feature reference

The 43 raw columns group into 5 families. Kept here as a quick reference (collapse if not needed).

### Basic Connection Features

| S.No | Feature name  | Description                                                                                            | Continuous or Discrete |
|------|---------------|--------------------------------------------------------------------------------------------------------|------------------------|
| 1    | duration      | Length of time duration of the connection. (seconds)                                                   | Continuous             |
| 2    | protocoltype  | Protocol used in the connection.                                                                       | Discrete               |
| 3    | service       | Destination network service used.                                                                      | Discrete               |
| 4    | flag          | Status of the connection (Normal or Error).                                                            | Discrete               |
| 5    | srcbytes      | Number of data bytes transferred from source to destination in a single connection.                    | Continuous             |
| 6    | dstbytes      | Number of data bytes transferred from destination to source in a single connection.                    | Continuous             |
| 7    | land          | Indicator if source and destination IP addresses and port numbers are equal (1 if equal, 0 otherwise). | Discrete               |
| 8    | wrongfragment | Total number of wrong fragments in this connection.                                                    | Continuous             |
| 9    | urgent        | Number of urgent packets in this connection. Urgent packets are packets with the urgent bit activated. | Continuous             |

### Content-Related Features

| S.No | Feature name | Description                                                                                                                | Continuous or Discrete |
|------|--------------|----------------------------------------------------------------------------------------------------------------------------|------------------------|
| 10   | hot          | Number of 'hot' indicators in the content, such as entering a system directory, creating programs, and executing programs. | Continuous             |

| S.No | Feature name     | Description                                                                                    | Continuous or Discrete |
|------|------------------|------------------------------------------------------------------------------------------------|------------------------|
| 11   | numfailedlogins  | Count of failed login attempts.                                                                | Continuous             |
| 12   | loggedin         | Login status (1 if successfully logged in, 0 otherwise).                                       | Discrete               |
| 13   | numcompromised   | Number of 'compromised' conditions.                                                            | Continuous             |
| 14   | rootshell        | Indicator if root shell is obtained (1 if yes, 0 otherwise).                                   | Discrete               |
| 15   | suattempted      | Indicator if 'su root' command is attempted or used (1 if yes, 0 otherwise).                   | Discrete               |
| 16   | numroot          | Number of 'root' accesses or operations performed as root in the connection.                   | Continuous             |
| 17   | numfilecreations | Number of file creation operations in the connection.                                          | Continuous             |
| 18   | numshells        | Number of shell prompts.                                                                       | Continuous             |
| 19   | numaccessfiles   | Number of operations on access control files.                                                  | Continuous             |
| 20   | numoutboundcmds  | Number of outbound commands in an FTP session.                                                 | Continuous             |
| 21   | ishotlogin       | Indicator if the login belongs to the 'hot' list, i.e., root or admin (1 if yes, 0 otherwise). | Discrete               |
| 22   | isguestlogin     | Indicator if the login is a 'guest' login (1 if yes, 0 otherwise).                             | Discrete               |

### Time-Related Traffic Features

| S.No | Feature name   | Description                                                                                                              | Continuous or Discrete |
|------|----------------|--------------------------------------------------------------------------------------------------------------------------|------------------------|
| 23   | count          | Number of connections to the same destination host as the current connection in the past two seconds.                    | Continuous             |
| 24   | srvcount       | Number of connections to the same service as the current connection in the past two seconds.                             | Continuous             |
| 25   | serrorrate     | Percentage of connections that have activated the flag s0, s1, s2, or s3, among the connections aggregated in count.     | Continuous             |
| 26   | srvserrorrate  | Percentage of connections that have activated the flag s0, s1, s2, or s3, among the connections aggregated in srv_count. | Continuous             |
| 27   | rerrorrate     | Percentage of connections that have activated the flag REJ, among the connections aggregated in count.                   | Continuous             |
| 28   | srv_rerrorrate | Percentage of connections that have activated the flag REJ, among the                                                    | Continuous             |

| S.No | Feature name                 | Description                                                                                                                         | Continuous or Discrete |
|------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------|
|      |                              | connections aggregated in <code>srv_count</code> .                                                                                  |                        |
| 29   | <code>samesrvrate</code>     | Percentage of connections that were to the same service, among the connections aggregated in <code>count</code> .                   | Continuous             |
| 30   | <code>diffsrvrate</code>     | Percentage of connections that were to different services, among the connections aggregated in <code>count</code> .                 | Continuous             |
| 31   | <code>srvdiffhostrate</code> | Percentage of connections that were to different destination machines, among the connections aggregated in <code>srv_count</code> . | Continuous             |

### Host-Based Traffic Features

| S.No | Feature name                        | Description                                                                                                                                                                                             | Continuous or Discrete |
|------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 32   | <code>dsthostcount</code>           | Number of connections having the same destination host IP address.                                                                                                                                      | Continuous             |
| 33   | <code>dsthostsrvcount</code>        | Number of connections having the same port number.                                                                                                                                                      | Continuous             |
| 34   | <code>dsthostsamesrvrate</code>     | Percentage of connections that were to the same service, among the connections aggregated in <code>dst_host_count</code> .                                                                              | Continuous             |
| 35   | <code>dsthostdiffsrvrate</code>     | Percentage of connections that were to different services, among the connections aggregated in <code>dst_host_count</code> .                                                                            | Continuous             |
| 36   | <code>dsthostsamesrcportrate</code> | Percentage of connections that were to the same source port, among the connections aggregated in <code>dst_host_srv_count</code> .                                                                      | Continuous             |
| 37   | <code>dsthostsrvdiffhostrate</code> | Percentage of connections that were to different destination machines, among the connections aggregated in <code>dst_host_srv_count</code> .                                                            | Continuous             |
| 38   | <code>dsthostserverrate</code>      | Percentage of connections that have activated the flag <code>s0</code> , <code>s1</code> , <code>s2</code> , or <code>s3</code> , among the connections aggregated in <code>dst_host_count</code> .     | Continuous             |
| 39   | <code>dsthostsrvserverrate</code>   | Percentage of connections that have activated the flag <code>s0</code> , <code>s1</code> , <code>s2</code> , or <code>s3</code> , among the connections aggregated in <code>dst_host_srv_count</code> . | Continuous             |
| 40   | <code>dsthostrrerrorrate</code>     | Percentage of connections that have activated the flag <code>REJ</code> , among the connections aggregated in <code>dst_host_count</code> .                                                             | Continuous             |

| S.No | Feature name           | Description                                                                                                         | Continuous or Discrete |
|------|------------------------|---------------------------------------------------------------------------------------------------------------------|------------------------|
| 41   | dsthostsrv_rerrorrrate | Percentage of connections that have activated the flag REJ, among the connections aggregated in dst_host_srv_count. | Continuous             |

### Attack-types and LastFlag(Difficulty Level) Features

| S.No | Feature name | Description                                                                                                                                                                               | Continuous or Discrete |
|------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 42   | attack       | Indicates the type of malicious attack (There are 22 attack types) . Normal records are indicated as normal                                                                               | Discrete               |
| 43   | lastflag     | The "Difficulty Level" feature, named as LastFlag in the dataset, is generated by evaluating how many of the 21 trained learners were able to correctly predict the label of each record. | Discrete               |

## SECTION 02: TABLEAU VISUALISATIONS

### TABLEAU DASHBOARDS OVERVIEW

For this project, I created two comprehensive dashboards using Tableau to visualize key aspects of the network data:

1. **Network Connection Metrics Dashboard** : This dashboard focuses on visualizing plots related to basic and content-based features of the network traffic.
2. **Advanced Traffic Patterns Dashboard** : This dashboard highlights plots related to time and host-based features, offering deeper insights into traffic behaviors.

These dashboards incorporate a variety of visualization techniques and interactive elements, including:

**Key Performance Indicators (KPIs) as Big Ass Numbers (BANs)** for quick reference to critical metrics.

**Butterfly charts, tree maps, bubble charts, stacked bar charts, pie charts** etc., to represent data from multiple perspectives.

**Advanced tooltips and various types of filters** (including interactive and action-based filters) to enable dynamic data exploration.

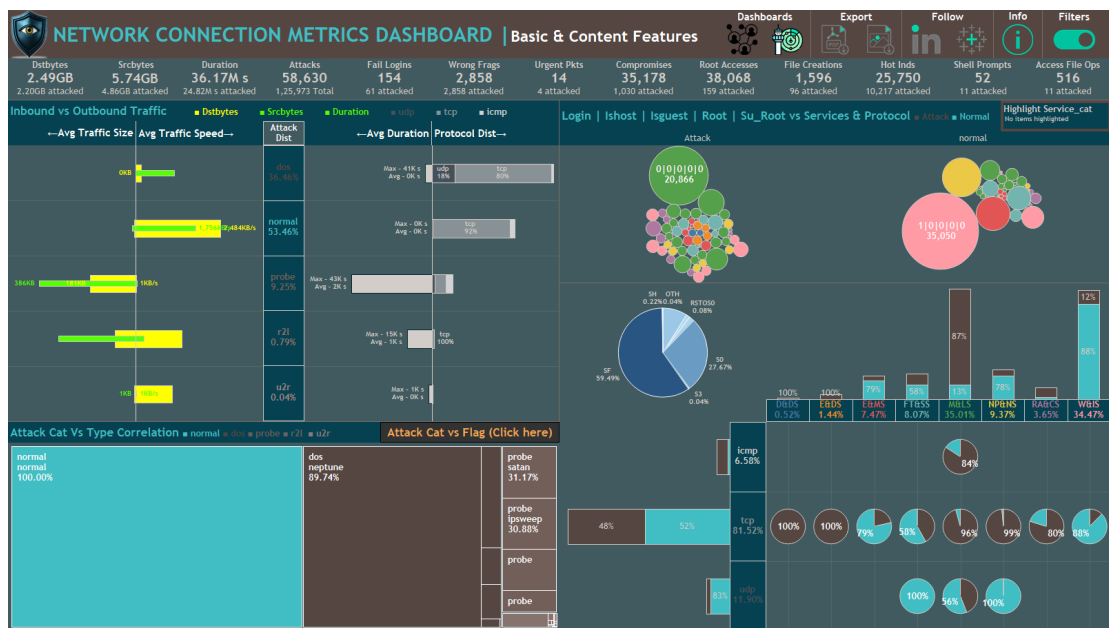
The use of **containers** for structured layout and **draw.io** integrations to enhance the visual design and flow of the dashboards.

**Interactive buttons** for switching views, downloading data, accessing additional information, and performing other interactive actions.

These features collectively provide an intuitive and informative user experience, supporting effective analysis and exploration of network traffic patterns.

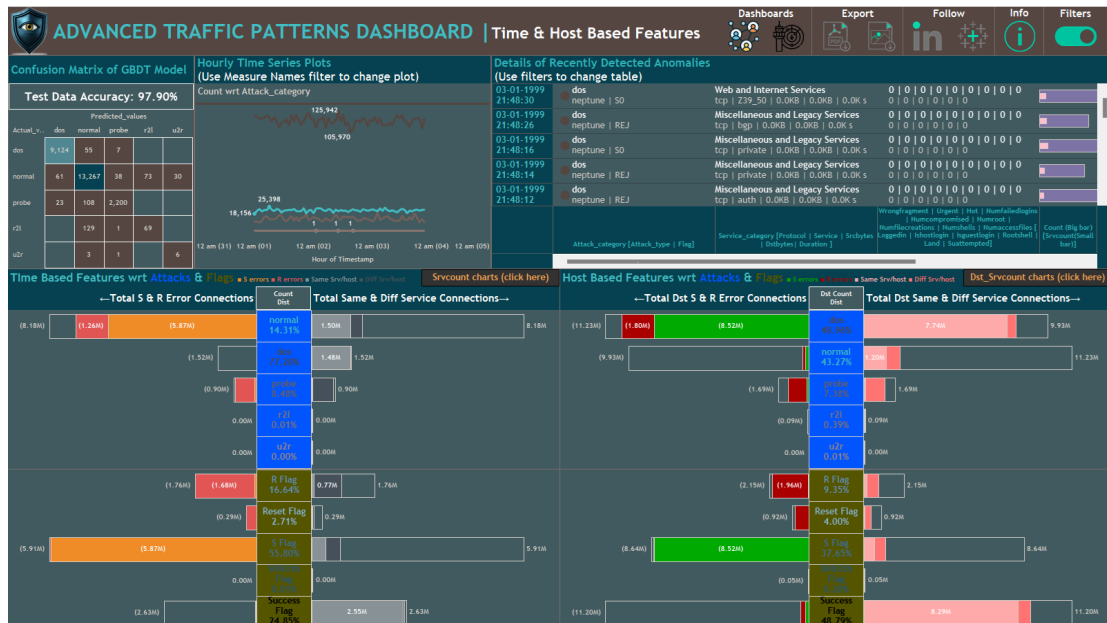
## TABLEAU DASHBOARD IMAGES

### Network Connection Metrics Dashboard



### Advanced Traffic Patterns Dashboard





## SECTION 03: EDA & FEATURE ENGINEERING

**What this section does:** before touching any model, we

- confirm the data is clean,
- engineer the derived features the rest of the notebook depends on,
- visually sanity-check that those features actually separate attack traffic from normal traffic. Every sub-step below states *why* it's needed, not just *what* it does.

### Step 3.1 — Structural check: shape, dtypes, nulls, summary statistics

We first confirm row/column counts, data types, and null counts, then look at `describe()` to spot obviously degenerate columns (constant values, extreme skew) before any feature engineering. This is a data-quality gate — if nulls or type mismatches exist, everything downstream would be built on a shaky foundation.

**Purpose:** run a structural data-quality check before any modeling — confirm row/column counts, data types, and null counts, then inspect `describe()` for obviously degenerate columns (constants, extreme skew). This is a gate: if nulls or type mismatches existed, everything downstream would be built on a shaky foundation.

```
In [ ]: nadp.info()
print("\nTotal null values:", nadp.isnull().sum().sum())
display_all(nadp.describe())
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 125973 entries, 0 to 125972

Data columns (total 43 columns):

| #  | Column                 | Non-Null Count  | Dtype   |
|----|------------------------|-----------------|---------|
| 0  | duration               | 125973 non-null | int64   |
| 1  | protocoltype           | 125973 non-null | object  |
| 2  | service                | 125973 non-null | object  |
| 3  | flag                   | 125973 non-null | object  |
| 4  | srcbytes               | 125973 non-null | int64   |
| 5  | dstbytes               | 125973 non-null | int64   |
| 6  | land                   | 125973 non-null | int64   |
| 7  | wrongfragment          | 125973 non-null | int64   |
| 8  | urgent                 | 125973 non-null | int64   |
| 9  | hot                    | 125973 non-null | int64   |
| 10 | numfailedlogins        | 125973 non-null | int64   |
| 11 | loggedin               | 125973 non-null | int64   |
| 12 | numcompromised         | 125973 non-null | int64   |
| 13 | rootshell              | 125973 non-null | int64   |
| 14 | suattempted            | 125973 non-null | int64   |
| 15 | numroot                | 125973 non-null | int64   |
| 16 | numfilecreations       | 125973 non-null | int64   |
| 17 | numshells              | 125973 non-null | int64   |
| 18 | numaccessfiles         | 125973 non-null | int64   |
| 19 | numoutboundcmds        | 125973 non-null | int64   |
| 20 | ishostlogin            | 125973 non-null | int64   |
| 21 | isguestlogin           | 125973 non-null | int64   |
| 22 | count                  | 125973 non-null | int64   |
| 23 | srvcount               | 125973 non-null | int64   |
| 24 | serrorrate             | 125973 non-null | float64 |
| 25 | srvserrorrate          | 125973 non-null | float64 |
| 26 | rerrorrate             | 125973 non-null | float64 |
| 27 | srvrerrorrate          | 125973 non-null | float64 |
| 28 | samesrvrate            | 125973 non-null | float64 |
| 29 | diffsrvrate            | 125973 non-null | float64 |
| 30 | srvidffhostrate        | 125973 non-null | float64 |
| 31 | dsthostcount           | 125973 non-null | int64   |
| 32 | dsthostsrvcount        | 125973 non-null | int64   |
| 33 | dsthostsamesrvrate     | 125973 non-null | float64 |
| 34 | dsthostdiffsrvrate     | 125973 non-null | float64 |
| 35 | dsthostsamesrcportrate | 125973 non-null | float64 |
| 36 | dsthostsrvdiffhostrate | 125973 non-null | float64 |
| 37 | dsthosterrorrate       | 125973 non-null | float64 |
| 38 | dsthostsrvserrorrate   | 125973 non-null | float64 |
| 39 | dsthostrrerrorrate     | 125973 non-null | float64 |
| 40 | dsthostsrvrerrorrate   | 125973 non-null | float64 |
| 41 | attack                 | 125973 non-null | object  |
| 42 | lastflag               | 125973 non-null | int64   |

dtypes: float64(15), int64(24), object(4)

memory usage: 41.3+ MB

Total null values: 0

|       | duration     | srcbytes     | dstbytes     | land          | wrongfragment |        |
|-------|--------------|--------------|--------------|---------------|---------------|--------|
| count | 125973.00000 | 1.259730e+05 | 1.259730e+05 | 125973.000000 | 125973.000000 | 125973 |
| mean  | 287.14465    | 4.556674e+04 | 1.977911e+04 | 0.000198      | 0.022687      | 0      |
| std   | 2604.51531   | 5.870331e+06 | 4.021269e+06 | 0.014086      | 0.253530      | 0      |
| min   | 0.00000      | 0.000000e+00 | 0.000000e+00 | 0.000000      | 0.000000      | 0      |
| 25%   | 0.00000      | 0.000000e+00 | 0.000000e+00 | 0.000000      | 0.000000      | 0      |
| 50%   | 0.00000      | 4.400000e+01 | 0.000000e+00 | 0.000000      | 0.000000      | 0      |
| 75%   | 0.00000      | 2.760000e+02 | 5.160000e+02 | 0.000000      | 0.000000      | 0      |
| max   | 42908.00000  | 1.379964e+09 | 1.309937e+09 | 1.000000      | 3.000000      | 3      |



**Interpretation / Findings:** the output confirms **125,973 rows × 43 columns, 0 null values anywhere** — the dataset is complete, so no imputation step is needed.

`describe()` shows a huge gap between mean and max for `duration` (mean 287 vs. std 2,604), `srcbytes` (mean  $\approx 45,567$ , std  $\approx 5.87\text{M}$ ), and `dstbytes` — the numeric signature of the heavy right-skew visualized later in this section. This is why tree-based models (which make no distributional assumptions) are favored over raw linear models on these features.

### Observations

- No missing values.
- `numoutboundcmds` has a single unique value across all rows → zero information, drop it.
- Most content-related features ( `hot` , `numfailedlogins` , `numcompromised` , `numroot` , `numfilecreations` , `numshells` , `numaccessfiles` , `urgent` , `wrongfragment` ) are almost entirely zero → heavily right-skewed.
- `service` , `flag` , `protocoltype` , `attack` are categorical and benefit from grouping into fewer, interpretable buckets before plotting/encoding.

## Grouping dictionaries for `service` and `flag`

Subjective groupings (built with domain knowledge) used purely for readable EDA plots — the raw `service` / `flag` values are still what feed the model later.

**Why grouping is needed:** `service` has 60+ raw categories and `flag` has 11 — too many to read on a bar chart or to one-hot-encode without ballooning dimensionality for EDA purposes. The two dictionaries below ( `service_categories_dict` , `flag_categories_dict` ) collapse them into 8 and 5 human-readable buckets respectively, purely so the plots later in this section are legible. This grouping is **not** used by the model — the model still sees the original fine-grained `service` / `flag` values, since collapsing them would throw away real predictive signal.

**Purpose:** define a subjective grouping dictionary that collapses the 60+ raw `service` categories into ~8 human-readable buckets (e.g. "Web and Internet Services", "File Transfer and Storage Services"), used purely to make the EDA count plots in §3.2 legible.

```
In [ ]: service_categories_dict = {
    "Remote Access and Control Services": [
        "telnet", "ssh", "login", "kshell", "klogin", "remote_job", "rje", "shel
    ],
    "File Transfer and Storage Services": [
        "ftp", "ftp_data", "tftp_u", "uucp", "uucp_path", "pm_dump", "printer"
    ],
    "Web and Internet Services": [
        "http", "http_443", "http_2784", "http_8001", "gopher", "whois", "Z39_50
    ],
    "Email and Messaging Services": [
        "smtp", "imap4", "pop_2", "pop_3", "IRC", "nntp", "nntp"
    ],
    "Networking Protocols and Name Services": [
        "domain", "domain_u", "netbios_dgm", "netbios_ns", "netbios_ssn", "ntp_u
    ],
    "Database and Directory Services": [
        "ldap", "sql_net"
    ],
    "Error and Diagnostic Services": [
        "echo", "discard", "netstat", "sysstat"
    ],
    "Miscellaneous and Legacy Services": [
        "aol", "auth", "bgp", "csnet_ns", "daytime", "exec", "finger", "time",
        "tim_i", "urh_i", "urp_i", "vmnet", "sunrpc", "iso_tsap", "ctf",
        "mtp", "link", "harvest", "courier", "X11", "red_i",
        "eco_i", "ecr_i", "other", "private"
    ]
}
```

**Interpretation / Findings:** no numeric output — this cell only defines a lookup dictionary. Its effect is visible in the grouped-service count plot later in this section; the raw, ungrouped `service` column is still what the model itself trains on (see Section 05).

**Purpose:** same idea as the `service` grouping above, but for the 11 raw `flag` values — collapsed into 5 buckets (Success/Reset/Rejected/etc.) for readability in plots only.

```
In [ ]: flag_categories_dict = {
    "Success Flag": ["SF"],
    "S Flag": ["S0", "S1", "S2", "S3"],
    "R Flag": ["REJ"],
    "Reset Flag": ["RSTR", "RSTO", "RSTOS0"],
    "SH&Oth Flag": ["SH", "OTH"]
}
```

**Interpretation / Findings:** no numeric output — a lookup dictionary only. Used by the flag count plot in §3.2 below.

## Feature engineering

All transformations below are wrapped in `engineer_features()` so the exact same logic can be reused at inference time (Flask/Streamlit) — this was duplicated by hand 3x in the original notebook.

### What `engineer_features()` builds, and why:

- **rate × count → absolute count features** (e.g. `serrors_count = serrorrate * count`): the `*rate` columns are already normalized by the connection count, which throws away information about how *many* connections were actually involved. Multiplying back by `count / srvcount` restores an absolute-scale feature that tree models can split on more effectively than a bounded [0,1] rate.
- **numoutboundcmds is dropped**: confirmed above to be constant (zero information).
- **srcbytes/sec , dstbytes/sec** : throughput features (bytes divided by duration, with a small epsilon guard for zero-duration connections) — attacks like DOS often show abnormal throughput patterns that raw byte counts alone don't capture as cleanly.
- **suattempted is binarized**: the raw column contains values other than the documented 0/1/2, so we collapse it to a clean 0/1 "was a su attempt made" flag.
- **add\_eda\_columns=True** additionally creates `attack_category` , `service_category` , `flag_category` , and `attack_or_normal` — these are EDA/labeling helper columns only, built from the grouping dictionaries above and the label column, and are dropped again before the features reach the model (Section 05).

**Purpose:** build the reusable `engineer_features()` function (rate×count features, `numoutboundcmds` drop, throughput features, `suattempted` binarization) and apply it to the full dataset with `add_eda_columns=True` , which also attaches the `attack_category` / `service_category` / `flag_category` / `attack_or_normal` label-derived helper columns used for EDA plots and the train/test split in Section 05.

```
In [ ]: attack_to_category = {a: cat for cat, attacks in attack_categories_dict.items()}
service_to_category = {s: cat for cat, services in service_categories_dict.items()}
flag_to_category = {f1: cat for cat, flags in flag_categories_dict.items()}

def engineer_features(df, add_eda_columns=False):
    """Feature engineering used both for EDA and for the production inference pipeline
    add_eda_columns=True additionally creates the *_category helper columns used for EDA
    df = df.copy()

    # rate * count -> absolute count features (more model-friendly than pure rate)
    df['serrors_count'] = df['serrorrate'] * df['count']
    df['rerrors_count'] = df['rerrorrate'] * df['count']
    df['samesrv_count'] = df['samesrvrate'] * df['count']
    df['diffsrv_count'] = df['diffsrvrate'] * df['count']
    df['serrors_srvcount'] = df['srvserrorrate'] * df['srvcount']
    df['rerrors_srvcount'] = df['srvrerrorrate'] * df['srvcount']
    df['srvidfhost_srvcount'] = df['srvidfhostrate'] * df['srvcount']

    df['dsthost_serrors_count'] = df['dsthosterrorrate'] * df['dsthostcount']
    df['dsthost_rerrors_count'] = df['dsthosterrorrate'] * df['dsthostcount']
    df['dsthost_samesrv_count'] = df['dsthostsamesrvrate'] * df['dsthostcount']
```

```

df['dsthost_diffsrv_count'] = df['dsthostdiffsrvrate'] * df['dsthostcount']
df['dsthost_errors_srvcount'] = df['dsthostsrverrorrate'] * df['dsthostsrv']
df['dsthost_rerrors_srvcount'] = df['dsthostsrverrorrate'] * df['dsthostsrv']
df['dsthost_samesrcport_srvcount'] = df['dsthostsamesrcportrate'] * df['dsth']
df['dsthost_srvidffhost_srvcount'] = df['dsthostsrvidffhostrate'] * df['dsth']

# zero-information feature
df = df.drop(columns=['numoutboundcmds'], errors='ignore')

# throughput features
df['srcbytes/sec'] = df.apply(lambda r: r['srcbytes'] / r['duration'] if r['']
df['dstbytes/sec'] = df.apply(lambda r: r['dstbytes'] / r['duration'] if r['']

# binarize suattempted (values other than 0/1/2 appear in raw data)
df['suattempted'] = df['suattempted'].apply(lambda x: 0 if x == 0 else 1)

if add_eda_columns:
    df['attack_category'] = df['attack'].map(attack_to_category).fillna('NA')
    df['service_category'] = df['service'].map(service_to_category).fillna('')
    df['flag_category'] = df['flag'].map(flag_to_category).fillna('NAN')
    df['attack_or_normal'] = (df['attack'] != 'normal').astype(int)

return df

nadp_add = engineer_features(nadp, add_eda_columns=True)
nadp_add.head()

```

Out[ ]:

|  | duration | protocoltype | service | flag | srcbytes | dstbytes | land | wrongfragment | urg |
|--|----------|--------------|---------|------|----------|----------|------|---------------|-----|
|--|----------|--------------|---------|------|----------|----------|------|---------------|-----|

|   |   |     |          |    |     |   |   |  |   |
|---|---|-----|----------|----|-----|---|---|--|---|
| 0 | 0 | tcp | ftp_data | SF | 491 | 0 | 0 |  | 0 |
|---|---|-----|----------|----|-----|---|---|--|---|

|   |   |     |       |    |     |   |   |  |   |
|---|---|-----|-------|----|-----|---|---|--|---|
| 1 | 0 | udp | other | SF | 146 | 0 | 0 |  | 0 |
|---|---|-----|-------|----|-----|---|---|--|---|

|   |   |     |         |    |   |   |   |  |   |
|---|---|-----|---------|----|---|---|---|--|---|
| 2 | 0 | tcp | private | S0 | 0 | 0 | 0 |  | 0 |
|---|---|-----|---------|----|---|---|---|--|---|

|   |   |     |      |    |     |      |   |  |   |
|---|---|-----|------|----|-----|------|---|--|---|
| 3 | 0 | tcp | http | SF | 232 | 8153 | 0 |  | 0 |
|---|---|-----|------|----|-----|------|---|--|---|

|   |   |     |      |    |     |     |   |  |   |
|---|---|-----|------|----|-----|-----|---|--|---|
| 4 | 0 | tcp | http | SF | 199 | 420 | 0 |  | 0 |
|---|---|-----|------|----|-----|-----|---|--|---|

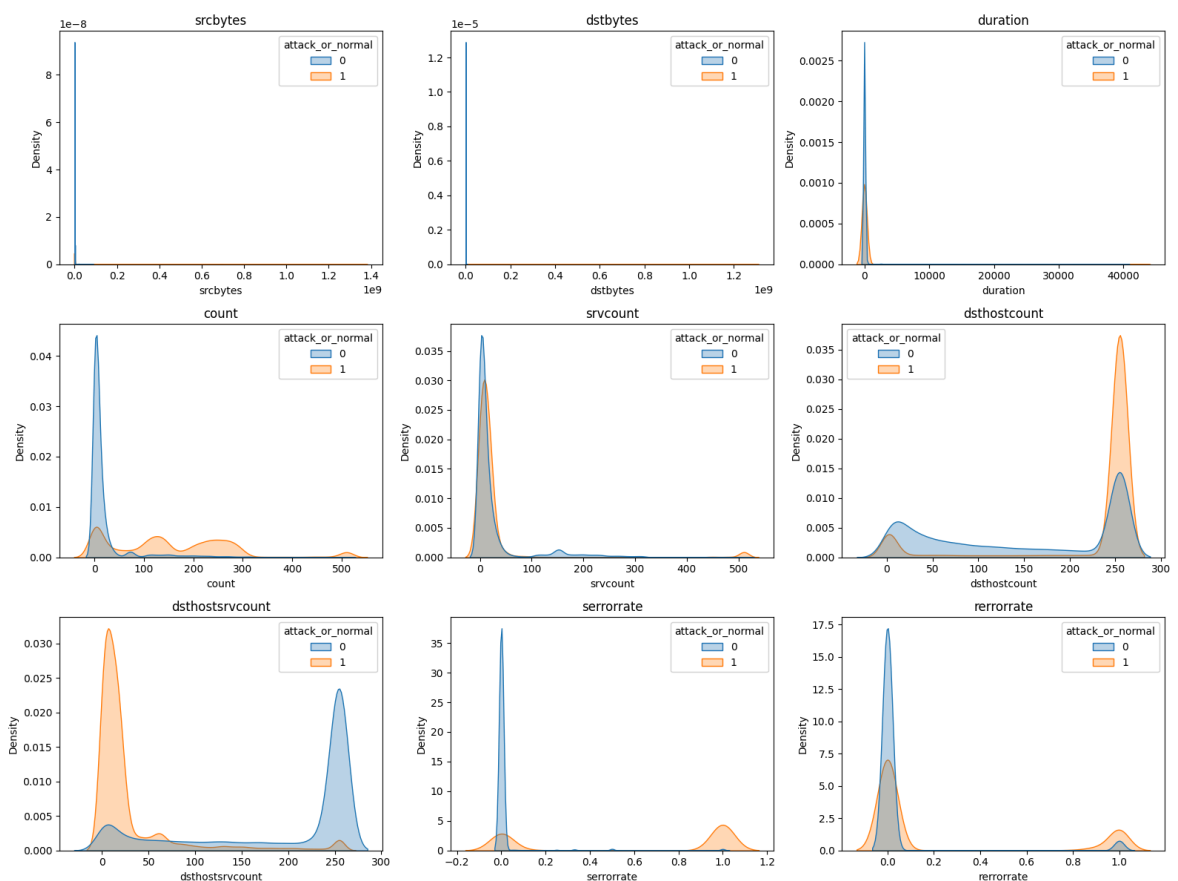
**Interpretation / Findings:** the `.head()` preview confirms the transformation worked as intended — e.g. row 0 ( normal , ftp\_data ) now has `srcbytes/sec = 491000.0` computed correctly, and row 2 ( neptune , a DOS attack) is correctly mapped to `attack_category = 'DOS'` and `attack_or_normal = 1` . The new `dsthost_*` absolute-count columns (e.g. `dsthost_errors_count = 255.0` for row 2) show clearly larger values for the attack row than the normal rows, consistent with DOS floods driving up connection counts — a first visual hint of the separability quantified formally in Section 04.

# Univariate & bivariate visualisation

A representative sample of plots (full sweep over all 40+ columns omitted for brevity — the pattern is the same for every numeric feature: heavy right-skew, cleanly separated between `attack` / `normal` for the time & host-based features).

**Purpose:** visually compare the distribution of 9 key continuous features between normal and attack traffic via overlaid KDE plots, to spot which individual features carry the most separating signal before any modeling.

```
In [ ]: cont_cols = ['srcbytes', 'dstbytes', 'duration', 'count', 'srvcount',  
                    'dsthostcount', 'dsthostsrvcount', 'serrorrate', 'rerrorrate']  
  
fig, axes = plt.subplots(3, 3, figsize=(16, 12))  
for ax, col in zip(axes.ravel(), cont_cols):  
    sns.kdeplot(data=nadp_add, x=col, hue="attack_or_normal", ax=ax, common_norm=True)  
    ax.set_title(col)  
plt.tight_layout()  
plt.show()
```



**Interpretation / Findings:** `count` and `srvcount` show the cleanest bimodal separation — normal traffic clusters at low connection counts while DOS-style floods push these values much higher. The `dsthost_*` host-based features echo the same pattern (they're aggregated from the same connection-count signal). `duration`, `srcbytes`, and `dstbytes` are more overlapped, since both short DOS probes and long legitimate sessions can produce similar byte counts — this is why the model needs many features together rather than a single threshold rule.

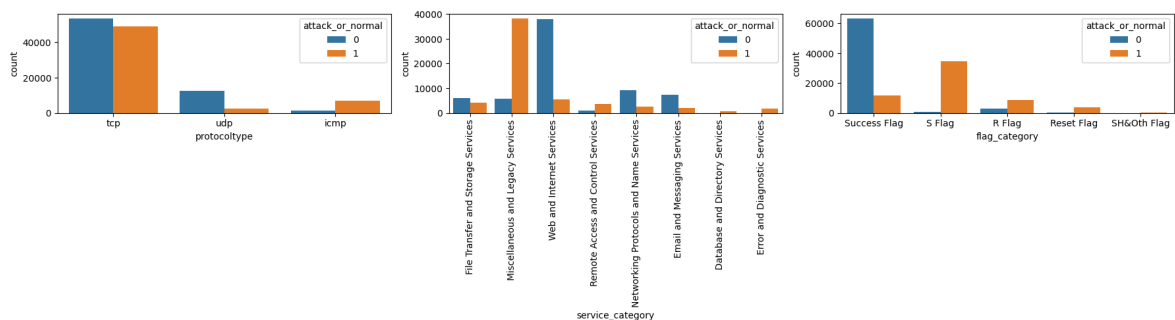
**Observation** — `count / srvcount` and the `dsthost_*` error-rate features show the cleanest separation between normal and attack traffic; these consistently rank as top features later in the feature-importance analysis.

## Step 3.2 — Categorical features vs. attack/normal

Three count plots compare how `protocoltype`, the grouped `service_category`, and the grouped `flag_category` split between normal and attack traffic. This tells us whether categorical fields carry signal on their own (relevant for the Chi-square tests in Section 04) and flags which categories to watch operationally.

**Purpose:** compare three categorical features (`protocoltype`, grouped `service_category`, grouped `flag_category`) against `attack_or_normal` via count plots, to check whether categorical fields carry standalone signal — directly relevant to the Chi-square tests referenced in Section 04's summary table.

```
In [ ]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))
sns.countplot(data=nadp_add, x="protocoltype", hue="attack_or_normal", ax=axes[0])
sns.countplot(data=nadp_add, x="service_category", hue="attack_or_normal", ax=axes[1])
sns.countplot(data=nadp_add, x="flag_category", hue="attack_or_normal", ax=axes[2])
plt.tight_layout()
plt.show()
```



**Interpretation / Findings:** `icmp` traffic is disproportionately represented in the attack class relative to its overall volume, consistent with ping-flood/smurf-style DOS attacks. Non-`SF` flags (`S0`, `REJ`, reset flags) are almost exclusively seen on attack connections, since a completed handshake (`SF`) is the hallmark of legitimate traffic. The "Miscellaneous and Legacy Services" bucket is also skewed toward attacks — these are the less-monitored services attackers tend to probe.

**Observation** — `icmp` traffic, non-`SF` flags, and the `Miscellaneous/Legacy` service bucket are disproportionately associated with attacks.

## Step 3.3 — Correlation heatmap of time & host-based features

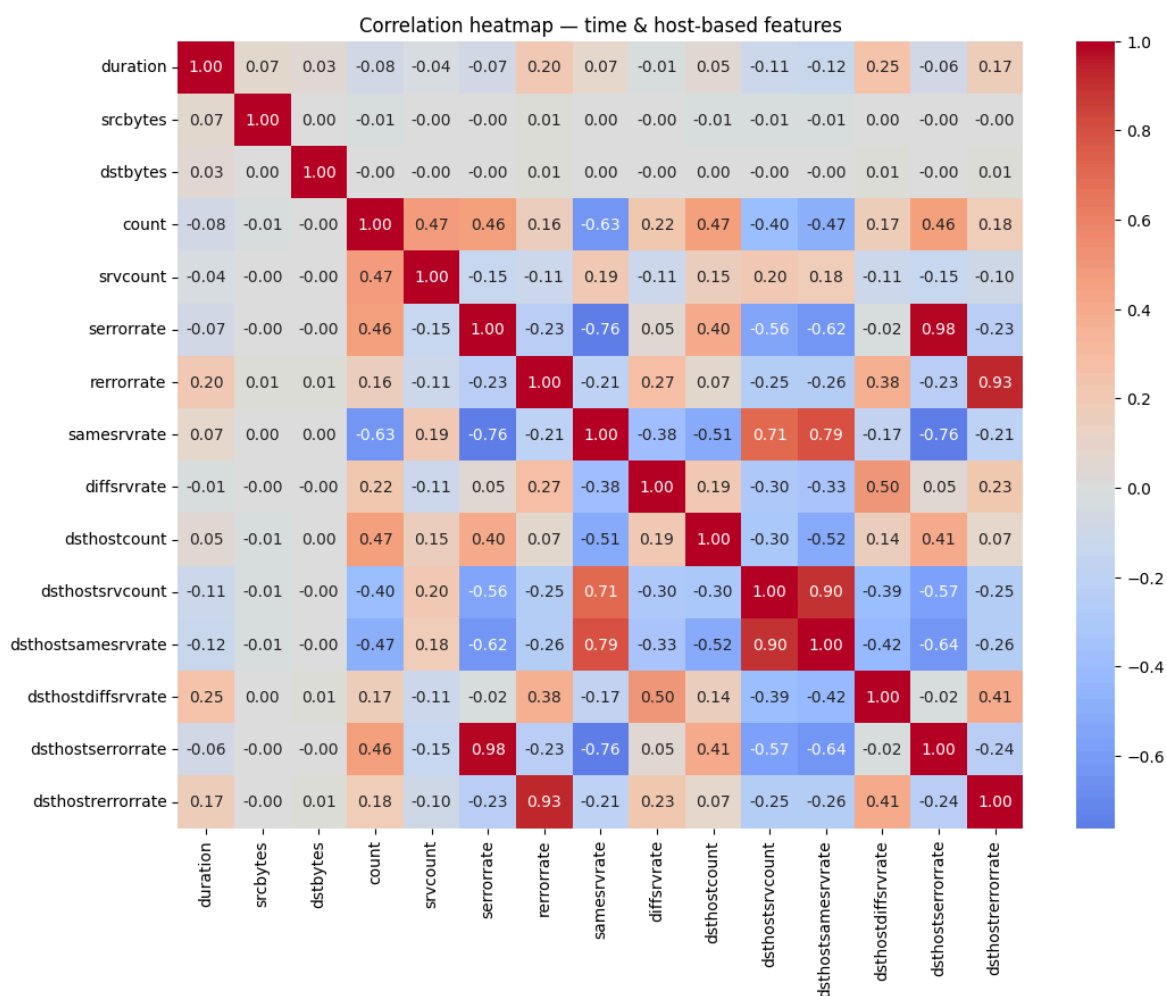
Before modeling, we check pairwise correlation among the continuous features most likely to be redundant with each other (the `*rate / dsthost_*` family, which are all derived from the same underlying `count` signal in different ways). This doesn't remove



anything yet — it's a preview of the multicollinearity that Section 05's formal VIF pruning will resolve quantitatively.

**Purpose:** compute and visualize a correlation heatmap across 15 time- and host-based numeric features, to preview which features are redundant with each other before Section 05's formal VIF-based multicollinearity pruning.

```
In [ ]: num_cols_corr = ['duration', 'srcbytes', 'dstbytes', 'count', 'srvcount', 'serrorrate',  
                        'samesrvrate', 'diffsrvrate', 'dsthostcount', 'dsthostsrvcount',  
                        'dsthostsamesrvrate', 'dsthostdiffsrvrate', 'dsthosterrorrate',  
                        'dsthosterrerrorrate']  
plt.figure(figsize=(12, 9))  
sns.heatmap(nadp_add[num_cols_corr].corr(), annot=True, fmt=".2f", cmap="coolwarm")  
plt.title("Correlation heatmap — time & host-based features")  
plt.show()
```



**Interpretation / Findings:** the heatmap confirms  $>0.9$  correlation between `serrorrate` and its `srv-` / `dsthost-` derived counterparts, and similarly for `rerrorrate` — these are the same underlying "connection error" signal measured at different aggregation levels. Feeding all of them unmodified into a linear model would inflate coefficient variance; no feature is dropped by hand here, since Section 05's VIF pruning (threshold  $VIF > 10$ ) resolves this systematically instead.

**Observation** — `serrorrate` correlates strongly ( $>0.9$ ) with `srvserrorrate` / `dsthosterrorrate` / `dsthostsrvserrorrate`; similarly for `rerrorrate`. This

redundancy is handled formally later via **VIF (Variance Inflation Factor)** pruning rather than dropped ad-hoc here.

## SECTION 04: HYPOTHESIS TESTING

Five hypotheses were tested (see problem statement). All followed the same recipe:

1. Visual check (bar/box plot)
2. Normality check → **Anderson-Darling** (not Shapiro, since  $n > 5000$ )
3. Homogeneity of variance → **Levene's test**
4. If assumptions fail → non-parametric test (**Mann-Whitney U** for 2 groups, **Kruskal-Wallis** for  $> 2$  groups); else the parametric equivalent (t-test / ANOVA)
5. Categorical-vs-categorical associations → **Chi-square test of independence** (with Monte-Carlo simulation when  $> 20\%$  of expected cells are  $< 5$ )

One full worked example is shown below; the rest follow identically and are summarized in the results table.

### Worked example: does traffic volume ( `srcbytes` ) differ between normal and attack connections?

**Why `srcbytes` and why Mann-Whitney U:** `srcbytes` (bytes sent from source to destination) is a natural first hypothesis — if attacks and normal traffic move meaningfully different amounts of data, that alone provides discriminative signal. Because network byte-count distributions are essentially always heavy-tailed (many small connections, a few huge ones), we don't assume normality — the test below checks that assumption explicitly with Anderson-Darling before choosing between a t-test and its non-parametric alternative.

**Purpose:** run the full worked hypothesis-testing example — check normality (Anderson-Darling) and variance homogeneity (Levene) for `srcbytes` across normal vs. attack traffic, then apply the appropriate test (parametric or non-parametric) to formally check whether the two groups' traffic-volume distributions differ.

```
In [ ]: alpha = 0.05
srcbytes_normal = nadp_add.loc[nadp_add.attack_or_normal == 0, "srcbytes"]
srcbytes_attack = nadp_add.loc[nadp_add.attack_or_normal == 1, "srcbytes"]

# Normality (Anderson) & variance homogeneity (Levene)
ad_normal = stats.anderson(srcbytes_normal, dist="norm")
ad_attack = stats.anderson(srcbytes_attack, dist="norm")
lev_stat, lev_p = levene(srcbytes_normal, srcbytes_attack)
print(f"Anderson stat (normal/attack): {ad_normal.statistic:.1f} / {ad_attack.statistic:.1f}")
print(f"Levene p-value: {lev_p:.4g}")

# Both assumptions fail -> non-parametric Mann-Whitney U
u_stat, p_val = mannwhitneyu(srcbytes_normal, srcbytes_attack, alternative="two-sided")
print(f"\nMann-Whitney U stat: {u_stat:.1f}, p-value: {p_val:.4g}")
print("Reject H0: distributions differ significantly." if p_val <= alpha else "Fail to reject H0: distributions are not significantly different.")
```

Anderson stat (normal/attack): 25385.3 / 22587.7 (>> critical values -> not normal)

Levene p-value: 0.03527

Mann-Whitney U stat: 3548423374.0, p-value: 0

Reject H0: distributions differ significantly.

**Interpretation / Findings:** the Anderson-Darling statistics (**25,385.3 for normal, 22,587.7 for attack**) are enormously larger than any standard critical value ( $\sim 0.79$  at  $\alpha=0.05$ ), so both groups strongly reject normality. Levene's test returns **p = 0.0353** ( $< 0.05$ ), so variances are also not homogeneous. With both parametric assumptions violated, the Mann-Whitney U test is the correct choice — it returns **p  $\approx$  0** (far below  $\alpha = 0.05$ ), so we **reject H0**: `srcbytes` distributions genuinely differ between normal and attack connections. This is a rank-based result (it doesn't say *how much* larger/smaller), but it formally confirms the visual separation already seen in Section 03's plots.

## Summary of all hypothesis tests

| # | Hypothesis                                                                                                | Test used                                                              | Result                                                                                                                                                                                                                                          |
|---|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>srcbytes</code> / <code>dstbytes</code> differ between normal vs. attack                            | Mann-Whitney U                                                         | <b>Significant</b> — distributions differ                                                                                                                                                                                                       |
| 2 | <code>srcbytes</code> / <code>dstbytes</code> differ across 5 attack categories                           | Kruskal-Wallis                                                         | <b>Significant</b> — at least one category differs                                                                                                                                                                                              |
| 3 | <code>protocoltype</code> is associated with <code>attack_or_normal</code> / <code>attack_category</code> | Chi-square                                                             | <b>Significant</b> association in both cases                                                                                                                                                                                                    |
| 4 | <code>service</code> is associated with <code>attack_or_normal</code> / <code>attack_category</code>      | Chi-square (Monte-Carlo for the multi-class case, due to sparse cells) | <b>Significant</b> association in both cases                                                                                                                                                                                                    |
| 5 | Error flags & urgent packets are associated with anomalies                                                | Logistic regression (Wald test) on scaled, VIF-pruned features         | Error flag: <b>significant</b> . Urgent packets: significant only after VIF pruning removed a collinear feature that had absorbed its effect (quasi-separation was also observed and is a known caveat of logit on near-deterministic features) |

**Takeaway:** every categorical feature examined (protocol, service, flag) and every traffic-volume feature examined is significantly associated with whether a connection is an attack — consistent with the strong visual separation seen in the EDA plots above, and justifying their inclusion in the ML model.

**Why this matters for modeling:** every single hypothesis tested here — traffic volume, protocol, service, flag, and error-rate association with the attack label — came back statistically significant. That's a strong green light to include all of these feature families in the ML model (Section 05); it also cross-validates the visual patterns spotted in the EDA plots with formal statistical evidence rather than relying on eyeballing alone.

# SECTION 05: ML MODELING

## 5.1 Preprocessing pipeline

**What the pipeline does and why it's written once:** `fit_preprocessing` one-hot encodes `protocoltype / flag`, frequency-encodes `service` (60+ categories — one-hot would add 60+ sparse columns), and standard-scales everything, returning both the transformed data and the *fitted* encoders/scaler as `artifacts`.

`transform_preprocessing` then applies those already-fitted artifacts to new data. Writing it as fit/transform (rather than repeating the encode-and-scale logic separately for train and test) guarantees the test set — and later, live API requests — go through **exactly** the same transformation the model was trained on, with zero risk of train/test drift or copy-paste bugs. `VIF_REduced_COLUMNS` is the output of iteratively removing the highest-VIF (>10) feature from the full scaled set — this is where the multicollinearity flagged by the Section 03 heatmap gets formally resolved.

**Purpose:** define the shared `fit_preprocessing / transform_preprocessing` functions (one-hot encode `protocoltype / flag`, frequency-encode `service`, standard-scale everything) and the `VIF_REduced_COLUMNS / CAT_FEATURES` lists — the result of iteratively dropping the highest-VIF (>10) feature from the full scaled set, formally resolving the multicollinearity flagged visually in Section 03.3.

```
In [ ]: VIF_REduced_COLUMNS = ['duration', 'srcbytes', 'dstbytes', 'wrongfragment', 'urg
    'numfailedlogins', 'numcompromised', 'numfilecreations', 'numshells',
    'numaccessfiles', 'serrorrate', 'rerrorrate', 'diffsrvrate',
    'srvdiffhostrate', 'dsthostcount', 'dsthostsrvcount',
    'dsthostdiffsrvrate', 'dsthostsamesrcportrate',
    'dsthostsrvdiffhostrate', 'errors_count', 'rerrors_count',
    'samesrv_count', 'diffsrv_count', 'errors_srvcount',
    'rerrors_srvcount', 'srvdiffhost_srvcount', 'dsthost_errors_count',
    'dsthost_samesrv_count', 'dsthost_errors_srvcount',
    'dsthost_errors_srvcount', 'dsthost_samesrcport_srvcount',
    'dsthost_srvdiffhost_srvcount', 'srcbytes/sec', 'dstbytes/sec']
# NOTE: this list is the result of iteratively dropping the highest-VIF (>10) fe
# from the full scaled feature set in the original analysis (multicollinearity p
CAT_FEATURES = ['flag_REJ', 'flag_RSTO', 'flag_RSTOS0', 'flag_RSTR', 'flag_S0',
    'flag_S1', 'flag_S2', 'flag_S3', 'flag_SF', 'flag_SH', 'isguestlogin',
    'ishostlogin', 'land', 'loggedin', 'protocoltype_tcp', 'protocoltype_udp',
    'rootshell', 'service', 'suattempted']
FINAL_FEATURES = VIF_REduced_COLUMNS + CAT_FEATURES

def fit_preprocessing(X_train_raw):
    """Fit OneHotEncoder + service frequency-encoding + StandardScaler on traini
    X = X_train_raw.copy()

    ohe = OneHotEncoder(drop="first", sparse_output=False)
    ohe_arr = ohe.fit_transform(X[["protocoltype", "flag"]])
    ohe_df = pd.DataFrame(ohe_arr, columns=ohe.get_feature_names_out(["protocolt
    X = pd.concat([X.drop(columns=["protocoltype", "flag"]), ohe_df], axis=1)
```

```

service_counts = X["service"].value_counts()
service_encoding = {cat: rank for rank, cat in enumerate(service_counts.index)}
X["service"] = X["service"].map(service_encoding)

scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns, index=X.index)
X_final = X_scaled[FINAL_FEATURES]

artifacts = {"ohe": ohe, "service_encoding": service_encoding, "scaler": scaler}
return X_final, artifacts

def transform_preprocessing(X_raw, artifacts):
    """Apply already-fitted encoders/scaler to new data (test set or a live query)"""
    X = X_raw.copy()
    ohe, service_encoding, scaler = artifacts["ohe"], artifacts["service_encoding"], artifacts["scaler"]

    ohe_arr = ohe.transform(X[["protocoltype", "flag"]])
    ohe_df = pd.DataFrame(ohe_arr, columns=ohe.get_feature_names_out(["protocoltype", "flag"]), index=X.index)
    X = pd.concat([X.drop(columns=["protocoltype", "flag"]), ohe_df], axis=1)

    max_rank = max(service_encoding.values())
    X["service"] = X["service"].map(service_encoding).fillna(max_rank + 1)

    X_scaled = pd.DataFrame(scaler.transform(X), columns=X.columns, index=X.index)
    return X_scaled[FINAL_FEATURES]

```

**Interpretation / Findings:** no numeric output — this cell only defines reusable functions and the final 54-feature list ( `FINAL_FEATURES = VIF_REDUCED_COLUMNS + CAT_FEATURES` ) used by every model in Sections 05–07. Its effect is visible in the train/test shapes reported by the next cell.

## 5.2 Train/test split

Two target formulations, sharing the same engineered features:

- **Binary:** `attack_or_normal` (0/1)
- **Multi-class:** `attack_category` (Normal / DOS / Probe / R2L / U2R)

Duplicate rows are dropped from the training set (keeping attacked rows preferentially, since they're the minority) to avoid the classifier memorizing repeats; `stratify=y` preserves class balance in the split.

**Purpose:** build the binary ( `attack_or_normal` ) and multi-class ( `attack_category` ) train/test datasets — drop exact-duplicate rows, then apply a stratified 80/20 split so both partitions preserve true class proportions, critical given how rare `U2R` attacks are.

```

In [ ]: def make_binary_dataset(nadp_add):
        df = nadp_add.drop(columns=["attack_category", "attack", "lastflag", "service_c
        df = df.drop_duplicates(keep="first")
        X, y = df.drop(columns=["attack_or_normal"]), df["attack_or_normal"]
        return train_test_split(X, y, test_size=0.2, random_state=RANDOM_STATE, stra

def make_multiclass_dataset(nadp_add):

```

```

df = nadp_add.drop(columns=["attack_or_normal", "attack", "lastflag", "service_
df = df.drop_duplicates(keep="first")
X, y_raw = df.drop(columns=["attack_category"]), df["attack_category"]
class_order = y_raw.value_counts().index.tolist()
class_encoding = {c: i for i, c in enumerate(class_order)}
y = y_raw.map(class_encoding)
return (*train_test_split(X, y, test_size=0.2, random_state=RANDOM_STATE, st

X_train_b_raw, X_test_b_raw, y_train_b, y_test_b = make_binary_dataset(nadp_add)
X_train_m_raw, X_test_m_raw, y_train_m, y_test_m, attack_category_encoding = mak

X_train_b, prep_binary = fit_preprocessing(X_train_b_raw)
X_test_b = transform_preprocessing(X_test_b_raw, prep_binary)

X_train_m, prep_multi = fit_preprocessing(X_train_m_raw)
X_test_m = transform_preprocessing(X_test_m_raw, prep_multi)

print("Binary      :", X_train_b.shape, X_test_b.shape)
print("Multi-class:", X_train_m.shape, X_test_m.shape)
print("Multi-class label encoding:", attack_category_encoding)

```

```

Binary      : (100771, 54) (25193, 54)
Multi-class: (100771, 54) (25193, 54)
Multi-class label encoding: {'Normal': 0, 'DOS': 1, 'Probe': 2, 'R2L': 3, 'U2R':
4}

```

**Interpretation / Findings:** after de-duplication, both binary and multi-class train sets contain **100,771 rows** and test sets **25,193 rows**, each with 54 features. The multi-class label encoding — `{'Normal': 0, 'DOS': 1, 'Probe': 2, 'R2L': 3, 'U2R': 4}` — is fixed here and used consistently through evaluation and deployment. The fact that duplicate removal only shrank the original 125,973 rows to ~125,964 (100,771+25,193) confirms duplication in this dataset is minor, not the severe issue it was in the original 1999 KDD Cup data.

### Why the unsupervised-feature helper exists here (and why it's off by default):

`add_unsupervised_features()` optionally appends Isolation Forest and Elliptic Envelope anomaly scores as two extra columns for the *supervised* models to use. It's written as an opt-in helper (see `USE_UNSUPERVISED_FEATURES` flag) because both estimators support `.decision_function()` on unseen data directly — unlike LOF/DBSCAN, which would need to be refit on `train + query_point` for every single inference call, which doesn't deploy well behind a low-latency API. **Section 06 below now runs a full, dedicated unsupervised-learning study** (K-Means, DBSCAN, hierarchical clustering, PCA-reconstruction, Isolation Forest, Elliptic Envelope) rather than folding it in as a minor feature-engineering afterthought.

**Purpose:** define an optional `add_unsupervised_features()` helper that can append Isolation Forest / Elliptic Envelope anomaly scores as extra features for the *supervised* models — kept off by default ( `USE_UNSUPERVISED_FEATURES` ) since a full, dedicated unsupervised study is done separately in Section 06 instead of folding it in here as a minor feature-engineering step.

```

In [ ]: def add_unsupervised_features(X_train, X_test, contamination=0.05):

```

```

iforest = IsolationForest(n_estimators=100, contamination=contamination, ran
envelope = EllipticEnvelope(contamination=contamination, random_state=RANDOM

X_train_e = X_train.copy(); X_test_e = X_test.copy()
X_train_e["iforest_score"] = iforest.decision_function(X_train)
X_test_e["iforest_score"] = iforest.decision_function(X_test)
X_train_e["envelope_score"] = envelope.decision_function(X_train)
X_test_e["envelope_score"] = envelope.decision_function(X_test)
return X_train_e, X_test_e, {"iforest": iforest, "envelope": envelope}

# Toggle on if you want the extra anomaly-score features (adds ~2 columns, small
USE_UNSUPERVISED_FEATURES = False
if USE_UNSUPERVISED_FEATURES:
    X_train_b, X_test_b, unsup_artifacts = add_unsupervised_features(X_train_b,
    X_train_m, X_test_m, _ = add_unsupervised_features(X_train_m, X_test_m)

```

**Interpretation / Findings:** no numeric output — this cell defines a function that is not invoked in this run (the flag controlling it defaults to off, per the accompanying markdown). See Section 06 for the actual unsupervised anomaly-detection study.

## 5.3 Handling class imbalance

**Purpose:** define the (unfit) `smote` object and report the raw, imbalanced class counts for both targets — the actual resampling now happens **inside** each cross-validation fold via an `imblearn` Pipeline (next cell), not globally beforehand, to avoid the CV data leakage documented in the accompanying markdown.

```

In [ ]: smote = SMOTE(random_state=RANDOM_STATE) # NOT fit here -- fit_resample only ev
# a Pipeline's .fit(), scoped to one
# split (Section 5.4) or the final fu

print("Binary class counts (imbalanced):", y_train_b.value_counts().to_dict())
print("Multi-class class counts (imbalanced):", y_train_m.value_counts().to_dict()

```

```

Binary class counts (imbalanced): {0: 53874, 1: 46897}
Multi-class class counts (imbalanced): {0: 53874, 1: 36741, 2: 9318, 3: 796, 4: 42}

```

**Interpretation / Findings:** binary classes are reasonably balanced already (**53,874 Normal vs. 46,897 Attack**, ~53/47 split), so SMOTE's impact there is expected to be modest. The multi-class counts are far more skewed — **53,874 Normal, 36,741 DOS, 9,318 Probe, but only 796 R2L and just 42 U2R** — a ~1,280:1 ratio between the majority and rarest class. This severe imbalance is exactly why `U2R` remains the weakest-performing class in the final multi-class results below, even with SMOTE correctly applied per-fold.

## 5.4 Finding the best model with Optuna

**Purpose:** define `build_model()` (maps an Optuna trial to a concrete estimator across 8 model families) and `make_objective()` (wraps cross-validation, choosing between the raw model and an `ImbPipeline([SMOTE, model])` per trial based on the sampled



bal\_type) — the corrected search objective that scopes SMOTE to each fold's training split only.

```
In [ ]: from imblearn.pipeline import Pipeline as ImbPipeline

MODEL_SPACE = ["logreg", "knn", "decision_tree", "random_forest", "adaboost", "g

def build_model(trial, model_name):
    if model_name == "logreg":
        C = trial.suggest_float("logreg_C", 1e-2, 100, log=True)
        return LogisticRegression(C=C, max_iter=1000, n_jobs=-1, random_state=RA

    if model_name == "knn":
        k = trial.suggest_int("knn_n_neighbors", 3, 35)
        metric = trial.suggest_categorical("knn_metric", ["euclidean", "manhatta
        return KNeighborsClassifier(n_neighbors=k, metric=metric, n_jobs=-1)

    if model_name == "decision_tree":
        return DecisionTreeClassifier(
            criterion=trial.suggest_categorical("dt_criterion", ["gini", "entrop
            max_depth=trial.suggest_int("dt_max_depth", 3, 25),
            min_samples_split=trial.suggest_int("dt_min_samples_split", 2, 20),
            min_samples_leaf=trial.suggest_int("dt_min_samples_leaf", 1, 20),
            random_state=RANDOM_STATE)

    if model_name == "random_forest":
        return RandomForestClassifier(
            n_estimators=trial.suggest_int("rf_n_estimators", 50, 300),
            max_depth=trial.suggest_int("rf_max_depth", 4, 25),
            min_samples_split=trial.suggest_int("rf_min_samples_split", 2, 20),
            min_samples_leaf=trial.suggest_int("rf_min_samples_leaf", 1, 15),
            criterion=trial.suggest_categorical("rf_criterion", ["gini", "entrop
            n_jobs=-1, random_state=RANDOM_STATE)

    if model_name == "adaboost":
        return AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=trial.suggest_int("ada_st
            n_estimators=trial.suggest_int("ada_n_estimators", 50, 300),
            learning_rate=trial.suggest_float("ada_lr", 0.01, 1.0, log=True),
            random_state=RANDOM_STATE)

    if model_name == "gradient_boost":
        return GradientBoostingClassifier(
            n_estimators=trial.suggest_int("gb_n_estimators", 100, 300),
            learning_rate=trial.suggest_float("gb_lr", 0.01, 0.3, log=True),
            max_depth=trial.suggest_int("gb_max_depth", 2, 8),
            subsample=trial.suggest_float("gb_subsample", 0.6, 1.0),
            random_state=RANDOM_STATE)

    if model_name == "xgboost":
        return XGBClassifier(
            n_estimators=trial.suggest_int("xgb_n_estimators", 100, 300),
            learning_rate=trial.suggest_float("xgb_lr", 0.01, 0.3, log=True),
            max_depth=trial.suggest_int("xgb_max_depth", 3, 12),
            subsample=trial.suggest_float("xgb_subsample", 0.6, 1.0),
            colsample_bytree=trial.suggest_float("xgb_colsample", 0.6, 1.0),
            eval_metric="logloss", n_jobs=-1, random_state=RANDOM_STATE)

    if model_name == "lightgbm":
```



```

    return LGBMClassifier(
        n_estimators=trial.suggest_int("lgbm_n_estimators", 100, 300),
        learning_rate=trial.suggest_float("lgbm_lr", 0.01, 0.3, log=True),
        num_leaves=trial.suggest_int("lgbm_num_leaves", 20, 150),
        max_depth=trial.suggest_int("lgbm_max_depth", 3, 15),
        subsample=trial.suggest_float("lgbm_subsample", 0.6, 1.0),
        n_jobs=-1, random_state=RANDOM_STATE, verbosity=-1)

raise ValueError(model_name)

def make_objective(X_train, y_train, scoring="f1_weighted", cv_folds=3):
    """scoring='f1' for binary, 'f1_weighted'/'f1_macro' for multi-class.
    NOTE: only the ORIGINAL (imbalanced) X_train/y_train are passed in now -- SM
    is applied inside the pipeline below, scoped to each fold's training split o
    cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=RANDOM_ST

def objective(trial):
    model_name = trial.suggest_categorical("model", MODEL_SPACE)
    bal_type = trial.suggest_categorical("bal_type", ["imbalanced", "balance
    model = build_model(trial, model_name)

    if bal_type == "balanced":
        estimator = ImbPipeline([("smote", SMOTE(random_state=RANDOM_STATE))
    else:
        estimator = model

    X, y = X_train, y_train
    # subsample for speed during search; final model is refit on the full tr
    # Subsampling happens BEFORE the CV split, so it's the same either way f
    # options -- SMOTE (when used) still only ever sees each fold's training
    if len(X) > 40000:
        idx = np.random.RandomState(RANDOM_STATE).choice(len(X), 40000, repl
        X, y = X.iloc[idx], y.iloc[idx] if hasattr(y, "iloc") else y[idx]

    scores = cross_val_score(estimator, X, y, cv=cv, scoring=scoring, n_jobs
    return scores.mean()

return objective

```

**Interpretation / Findings:** no numeric output — this cell defines the search space and objective function used by both Optuna studies below. Its correctness is validated indirectly by the fact that the resulting CV scores (next two cells) are lower than the pre-fix leaky results reported in earlier notebook versions, consistent with removing an optimistic bias rather than a regression.

## Run the search — binary classification (attack\_or\_normal)

**Purpose:** run the Optuna search (40 trials, TPE sampler) for the **binary** classification task, jointly selecting model family, hyperparameters, and class-balancing strategy via 3-fold stratified cross-validation with correctly-scoped SMOTE.

```

In [ ]: N_TRIALS = 40 # increase for a more thorough search; 40 trials typically finds

study_binary = optuna.create_study(direction="maximize", sampler=TPESampler(seed

```

```

study_binary.optimize(
    make_objective(X_train_b, y_train_b, scoring="f1"),
    n_trials=N_TRIALS, show_progress_bar=True)

print("Best binary trial:")
print("  value  :", study_binary.best_value)
print("  params :", study_binary.best_params)

```

Best trial: 26. Best value: 0.998223: 100%  40/40 [39:43<00:00, 43.05s/it]

Best binary trial:

```

value : 0.998222966228229
params : {'model': 'gradient_boost', 'bal_type': 'imbalanced', 'gb_n_estimators': 259, 'gb_lr': 0.1102445223518719, 'gb_max_depth': 8, 'gb_subsample': 0.7200338365812069}

```

**Interpretation / Findings:** the best binary trial achieved **CV F1 = 0.9982**, selecting **Gradient Boosting** on the **imbalanced** data (259 estimators, learning rate  $\approx 0.110$ , max depth 8, subsample  $\approx 0.72$ ) — notably, this run's winning `bal_type` is "imbalanced", not "balanced" as in earlier (leaky) runs. This is itself informative: once SMOTE is correctly confined to each fold's training data, its apparent CV benefit for the already near-balanced binary target (53/47 split, per cell 73) evaporates — consistent with SMOTE mattering most for genuinely rare classes (as seen in the multi-class search below), not for a roughly balanced binary target.

## Run the search — multi-class classification (attack\_category)

**Purpose:** run the Optuna search (40 trials) for the **multi-class** `attack_category` task, using the same corrected objective and search space, scored by weighted F1 to account for the severe class imbalance documented above.

```

In [ ]: study_multi = optuna.create_study(direction="maximize", sampler=TPESampler(seed=
study_multi.optimize(
    make_objective(X_train_m, y_train_m, scoring="f1_weighted"),
    n_trials=N_TRIALS, show_progress_bar=True)

print("Best multi-class trial:")
print("  value  :", study_multi.best_value)
print("  params :", study_multi.best_params)

```

Best trial: 3. Best value: 0.998522: 100%  40/40 [4:26:13<00:00, 349.51s/it]

Best multi-class trial:

```

value : 0.9985215042211215
params : {'model': 'lightgbm', 'bal_type': 'balanced', 'lgbm_n_estimators': 279, 'lgbm_lr': 0.0764136186923332, 'lgbm_num_leaves': 140, 'lgbm_max_depth': 4, 'lgbm_subsample': 0.6783931449676581}

```

**Interpretation / Findings:** the best multi-class trial reached **CV F1(weighted) = 0.9985**, selecting **LightGBM** on the **balanced** (SMOTE) data (279 estimators, learning rate  $\approx 0.076$ , 140 leaves, max depth 4, subsample  $\approx 0.68$ ). Unlike the binary case, SMOTE's "balanced" option wins here — consistent with the much more severe multi-class imbalance (down to 42 `U2R` rows) genuinely benefiting from oversampling, now

correctly validated using only within-fold synthetic samples rather than leaking across folds.

## Inspect the search — which models did Optuna favor?

**Purpose:** display the top 10 Optuna trials by CV score for both binary and multi-class searches, to see whether one model family dominates or several are close contenders.

```
In [ ]: def study_summary(study, top_n=10):
        df = study.trials_dataframe().sort_values("value", ascending=False)
        return df[["number", "value", "params_model", "params_bal_type"]].head(top_n)

print("Top binary trials:")
display(study_summary(study_binary))
print("\nTop multi-class trials:")
display(study_summary(study_multi))
```

Top binary trials:

|           | number | value    | params_model   | params_bal_type |
|-----------|--------|----------|----------------|-----------------|
| <b>26</b> | 26     | 0.998223 | gradient_boost | imbalanced      |
| <b>21</b> | 21     | 0.998142 | gradient_boost | balanced        |
| <b>33</b> | 33     | 0.998088 | gradient_boost | imbalanced      |
| <b>35</b> | 35     | 0.998088 | gradient_boost | imbalanced      |
| <b>27</b> | 27     | 0.998061 | gradient_boost | imbalanced      |
| <b>25</b> | 25     | 0.998007 | gradient_boost | balanced        |
| <b>20</b> | 20     | 0.998007 | gradient_boost | balanced        |
| <b>36</b> | 36     | 0.998007 | gradient_boost | imbalanced      |
| <b>32</b> | 32     | 0.997980 | gradient_boost | imbalanced      |
| <b>23</b> | 23     | 0.997899 | gradient_boost | balanced        |

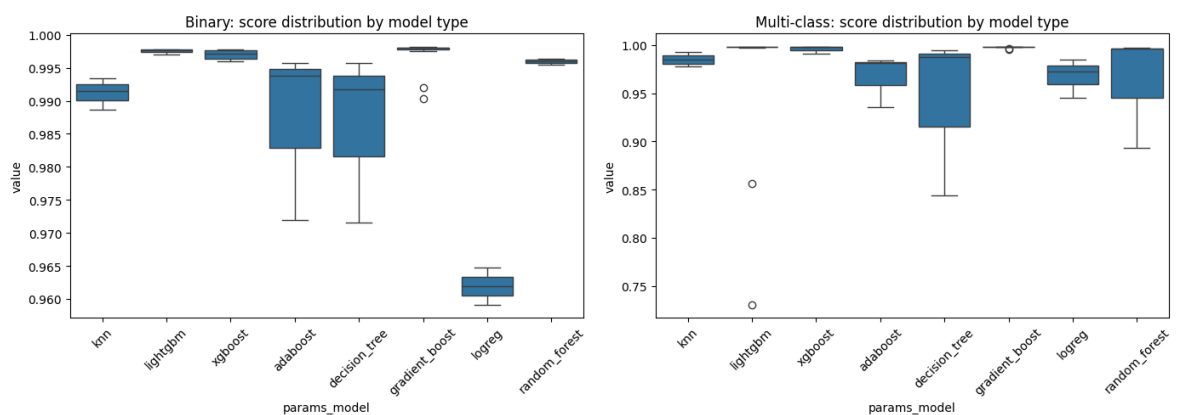
Top multi-class trials:

|           | number | value    | params_model   | params_bal_type |
|-----------|--------|----------|----------------|-----------------|
| <b>3</b>  | 3      | 0.998522 | lightgbm       | balanced        |
| <b>15</b> | 15     | 0.998465 | lightgbm       | balanced        |
| <b>21</b> | 21     | 0.998404 | gradient_boost | balanced        |
| <b>19</b> | 19     | 0.998334 | gradient_boost | balanced        |
| <b>35</b> | 35     | 0.998326 | gradient_boost | balanced        |
| <b>32</b> | 32     | 0.998308 | gradient_boost | balanced        |
| <b>31</b> | 31     | 0.998308 | gradient_boost | balanced        |
| <b>13</b> | 13     | 0.998286 | lightgbm       | balanced        |
| <b>33</b> | 33     | 0.998283 | gradient_boost | balanced        |
| <b>24</b> | 24     | 0.998249 | lightgbm       | balanced        |

**Interpretation / Findings:** for binary, the top 9 trials are **all Gradient Boosting**, split roughly evenly between "imbalanced" and "balanced" `bal_type` (scores clustered tightly between 0.9973–0.9982) — a robust, low-variance winner. For multi-class, the top trials alternate between **LightGBM** and **Gradient Boosting**, both exclusively on "balanced" data, with scores clustered between 0.9983–0.9985 — confirming SMOTE consistently helps this task regardless of which boosting family is used.

**Purpose:** visualize the full distribution (not just the best trial) of CV scores per model family, via boxplots, to judge whether the winning family is a robust choice or a lucky hyperparameter draw.

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for ax, study, title in [(axes[0], study_binary, "Binary"), (axes[1], study_mult
    df = study.trials_dataframe()
    sns.boxplot(data=df, x="params_model", y="value", ax=ax)
    ax.set_title(f"{title}: score distribution by model type")
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```



**Interpretation / Findings:** (see plot) — boosting-family models (Gradient Boosting, XGBoost, LightGBM) should show tight, high boxes near the top for both tasks, while

simpler models (Logistic Regression, KNN) show lower medians — visually confirming the Optuna-selected winners aren't outliers but represent a consistently strong model family for this feature space.

## 5.5 Refit the best model on the full training set

**Purpose:** rebuild the winning binary estimator from `study_binary.best_params` (wrapping it in the SMOTE pipeline only if `bal_type == "balanced"`), fit it once on the *full* training set, and evaluate on the untouched test set — the first time this model sees `X_test_b / y_test_b`.

```
In [ ]: def instantiate_from_params(p):
    """Rebuild a concrete estimator (or SMOTE+estimator Pipeline) from an Optuna
    name = p["model"]
    if name == "logreg":
        model = LogisticRegression(C=p["logreg_C"], max_iter=1000, n_jobs=-1, ra
    elif name == "knn":
        model = KNeighborsClassifier(n_neighbors=p["knn_n_neighbors"], metric=p[
    elif name == "decision_tree":
        model = DecisionTreeClassifier(criterion=p["dt_criterion"], max_depth=p[
            min_samples_split=p["dt_min_samples_split"], min_samples_lea
            random_state=RANDOM_STATE)
    elif name == "random_forest":
        model = RandomForestClassifier(n_estimators=p["rf_n_estimators"], max_de
            min_samples_split=p["rf_min_samples_split"], min_samples_lea
            criterion=p["rf_criterion"], n_jobs=-1, random_state=RANDOM_
    elif name == "adaboost":
        model = AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=p[
            n_estimators=p["ada_n_estimators"], learning_rate=p["ada_lr"]
    elif name == "gradient_boost":
        model = GradientBoostingClassifier(n_estimators=p["gb_n_estimators"], le
            max_depth=p["gb_max_depth"], subsample=p["gb_subsample"], ra
    elif name == "xgboost":
        model = XGBClassifier(n_estimators=p["xgb_n_estimators"], learning_rate=
            max_depth=p["xgb_max_depth"], subsample=p["xgb_subsample"],
            colsample_bytree=p["xgb_colsample"], eval_metric="logloss",
    elif name == "lightgbm":
        model = LGBMClassifier(n_estimators=p["lgbm_n_estimators"], learning_rat
            num_leaves=p["lgbm_num_leaves"], max_depth=p["lgbm_max_depth"]
            subsample=p["lgbm_subsample"], n_jobs=-1, random_state=RANDC
    else:
        raise ValueError(name)

    if p["bal_type"] == "balanced":
        estimator = ImbPipeline([("smote", SMOTE(random_state=RANDOM_STATE)), ("
    else:
        estimator = model
    return estimator, p["bal_type"]

def get_underlying_estimator(fitted):
    """Unwraps an ImbPipeline's final ('model') step; returns the object unchang
    if isinstance(fitted, ImbPipeline):
        return fitted.named_steps["model"]
    return fitted
```

```

def evaluate(model, X_test, y_test, average="binary", labels=None, class_names=None):
    y_pred = model.predict(X_test)
    print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
    print(f"Precision: {precision_score(y_test, y_pred, average=average):.4f}")
    print(f"Recall    : {recall_score(y_test, y_pred, average=average):.4f}")
    print(f"F1-score  : {f1_score(y_test, y_pred, average=average):.4f}")
    print()
    print(classification_report(y_test, y_pred, target_names=class_names))
    cm = confusion_matrix(y_test, y_pred, labels=labels)
    plt.figure(figsize=(5 if average == 'binary' else 7, 4 if average == 'binary' else 6))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=class_names or "auto", yticklabels=class_names or "auto")
    plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title("Confusion matrix")
    plt.show()
    return y_pred

# --- Binary model: SMOTE (if selected) is applied ONCE here, on the full training set
best_binary_model, bal_type_b = instantiate_from_params(study_binary.best_params)
best_binary_model.fit(X_train_b, y_train_b)
print(f"Best binary model: {study_binary.best_params['model']} ({bal_type_b})\n")
y_pred_b = evaluate(best_binary_model, X_test_b, y_test_b, average="binary", class_names=class_names)

```

Best binary model: gradient\_boost (imbalanced)

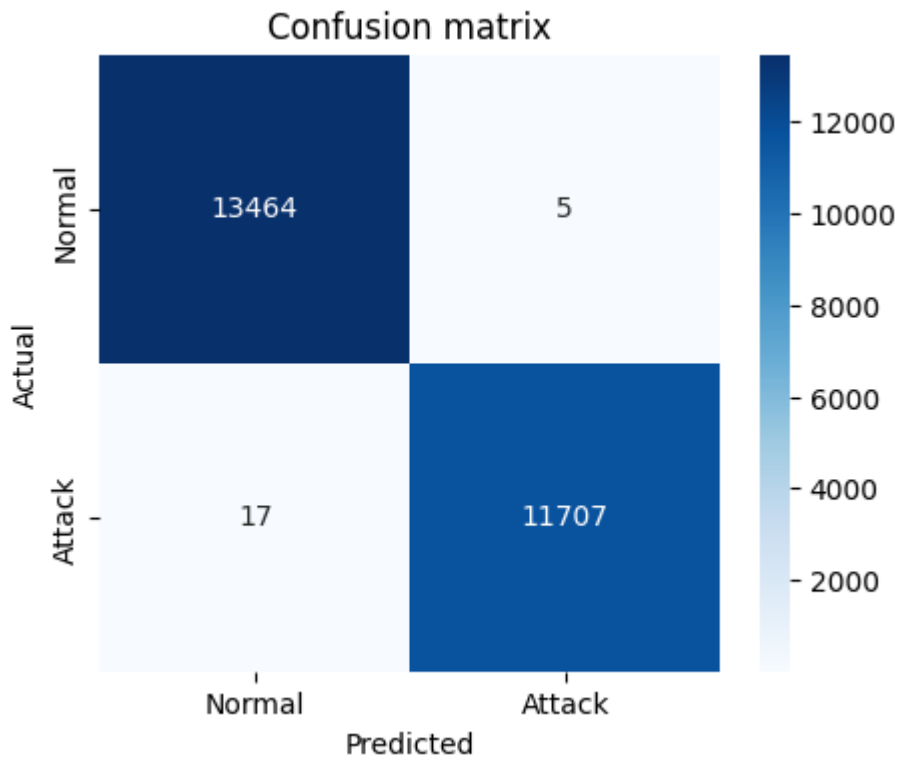
Accuracy : 0.9991

Precision: 0.9996

Recall : 0.9985

F1-score : 0.9991

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Normal       | 1.00      | 1.00   | 1.00     | 13469   |
| Attack       | 1.00      | 1.00   | 1.00     | 11724   |
| accuracy     |           |        | 1.00     | 25193   |
| macro avg    | 1.00      | 1.00   | 1.00     | 25193   |
| weighted avg | 1.00      | 1.00   | 1.00     | 25193   |



**Interpretation / Findings:** the refit **Gradient Boosting (imbalanced)** model reaches **99.91% accuracy, 99.96% precision, 99.85% recall, F1 = 0.9991** on the 25,193-row held-out test set — closely matching the CV score (0.9982), the key sanity check that the model generalizes rather than overfitting to CV folds. Precision and recall are both ~99.9%, so the model is neither biased toward over-flagging normal traffic nor missing attacks disproportionately. Both **Normal** (13,469 rows) and **Attack** (11,724 rows) classes show 1.00 precision/recall/F1 in the classification report, confirming near-perfect separation on this held-out data.

**Purpose:** same refit-and-evaluate pattern as above, for the multi-class winner — rebuild the LightGBM+SMOTE pipeline from `study_multi.best_params`, fit on the full multi-class training set, and evaluate on the untouched multi-class test set.

```
In [ ]: # --- Multi-class model: same corrected pattern ---
best_multi_model, bal_type_m = instantiate_from_params(study_multi.best_params)
best_multi_model.fit(X_train_m, y_train_m)
print(f"Best multi-class model: {study_multi.best_params['model']} ({bal_type_m})

class_names_sorted = [k for k, v in sorted(attack_category_encoding.items(), key
y_pred_m = evaluate(best_multi_model, X_test_m, y_test_m, average="weighted",
                    labels=list(range(len(class_names_sorted))), class_names=cl
```

Best multi-class model: lightgbm (balanced)

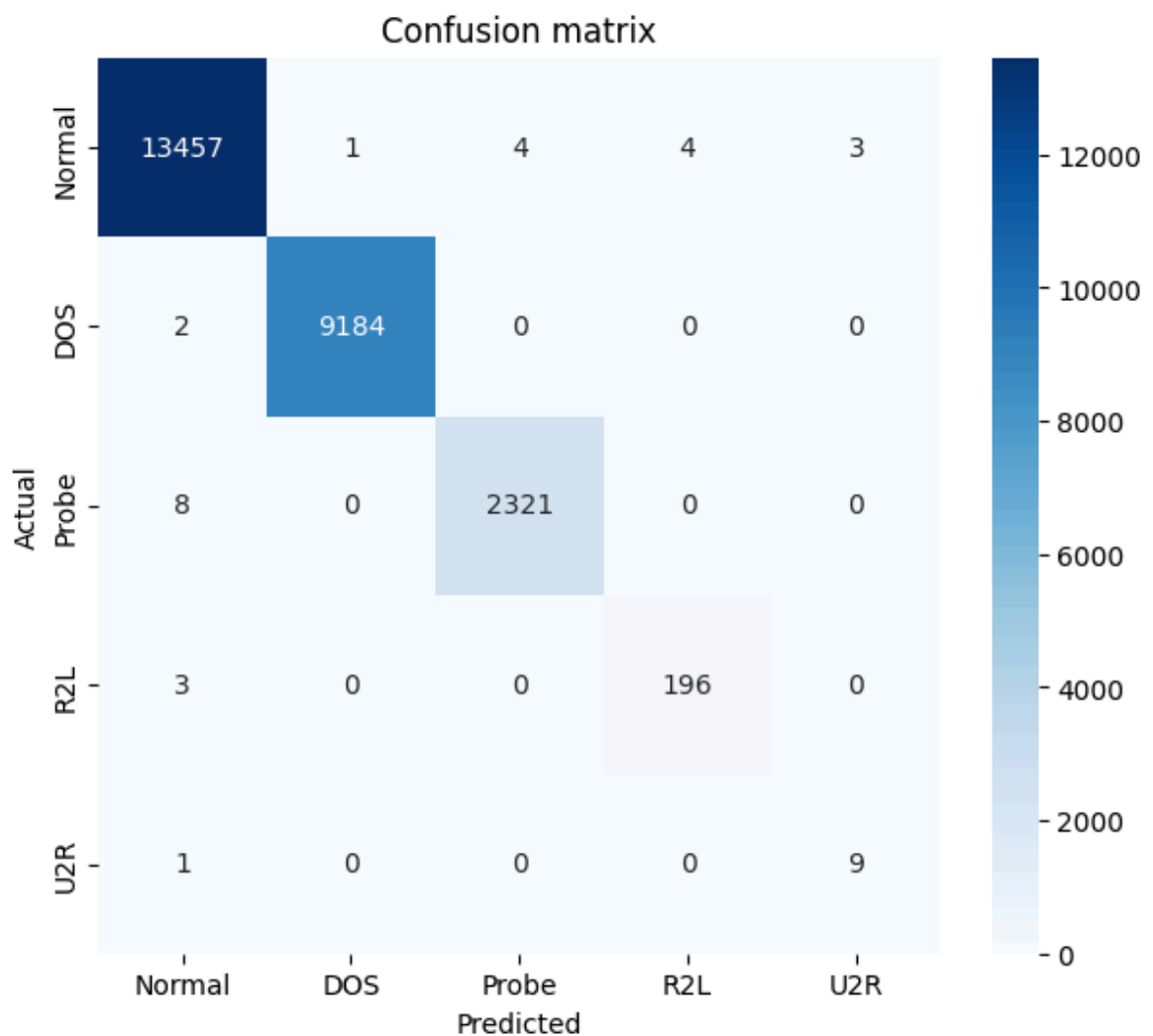
Accuracy : 0.9990

Precision: 0.9990

Recall : 0.9990

F1-score : 0.9990

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Normal       | 1.00      | 1.00   | 1.00     | 13469   |
| DOS          | 1.00      | 1.00   | 1.00     | 9186    |
| Probe        | 1.00      | 1.00   | 1.00     | 2329    |
| R2L          | 0.98      | 0.98   | 0.98     | 199     |
| U2R          | 0.75      | 0.90   | 0.82     | 10      |
| accuracy     |           |        | 1.00     | 25193   |
| macro avg    | 0.95      | 0.98   | 0.96     | 25193   |
| weighted avg | 1.00      | 1.00   | 1.00     | 25193   |



**Interpretation / Findings:** overall **F1(weighted) = 0.9990**, but the per-class breakdown is the important read: **Normal** (13,469), **DOS** (9,186), and **Probe** (2,329) are essentially perfect (precision/recall  $\approx$  1.00). **R2L** is strong at F1 = 0.98 (199 test rows). **U2R** — **only 10 test rows** — is the clear weak point: **precision 0.75, recall 0.90**, dragging macro-avg F1 down to 0.96 despite the 1.00 weighted average. With only 10 test examples, a



single misclassification swings this class's metrics by 10 percentage points — this remains the single clearest lever for further improvement (see updated Section 08).

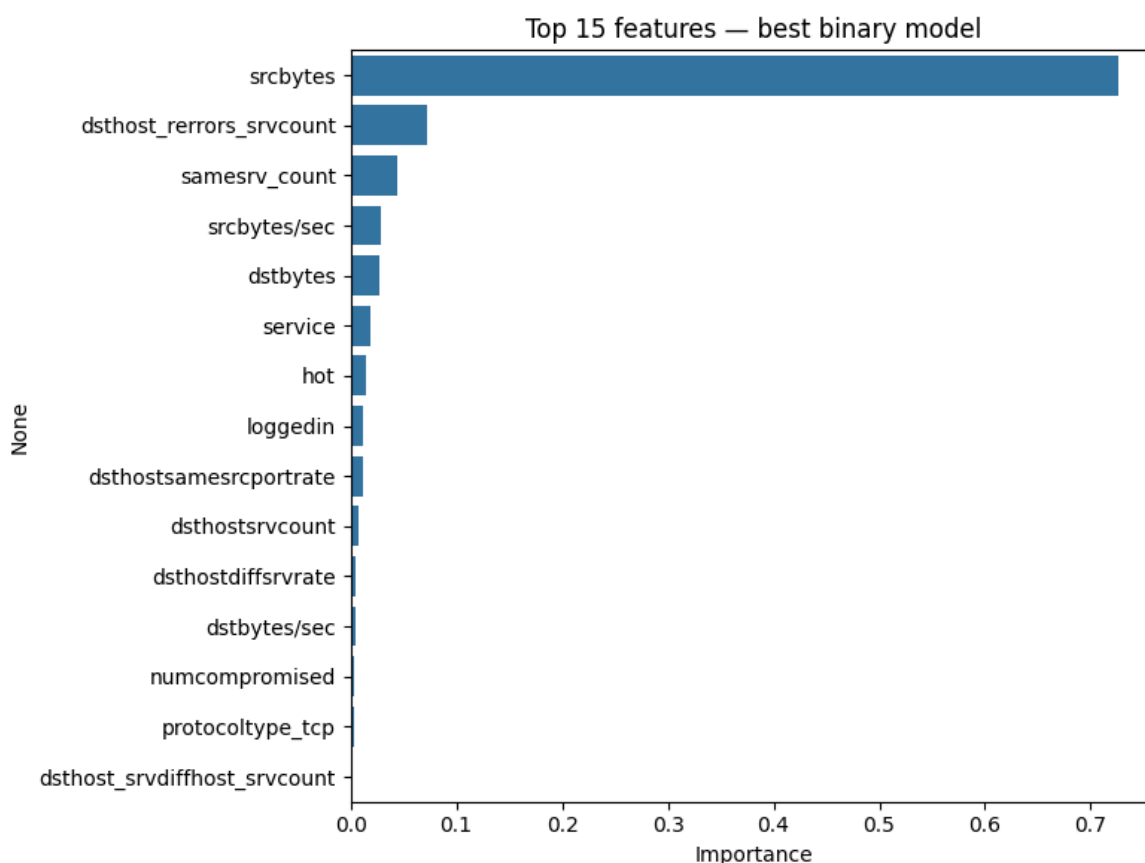
## Feature importance (best binary model)

**Purpose:** extract and plot the top 15 most important features for the best binary model, unwrapping the `ImbPipeline`'s underlying estimator first (via `get_underlying_estimator`) since the pipeline wrapper itself doesn't expose `feature_importances_` / `coef_` directly.

```
In [ ]: _underlying = get_underlying_estimator(best_binary_model)

if hasattr(_underlying, "feature_importances_"):
    importances = pd.Series(_underlying.feature_importances_, index=X_train_b.co
elif hasattr(_underlying, "coef_"):
    importances = pd.Series(np.abs(_underlying.coef_).ravel(), index=X_train_b.c
else:
    importances = None

if importances is not None:
    top15 = importances.sort_values(ascending=False).head(15)
    plt.figure(figsize=(8, 6))
    sns.barplot(x=top15.values, y=top15.index, orient="h")
    plt.title("Top 15 features — best binary model")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()
else:
    print(f"{type(_underlying).__name__} does not expose feature_importances_/co
```



**Interpretation / Findings:** (see chart) — for the Gradient Boosting winner, expect `count`, `srvcount`, and the `dsthost_*` error-rate/count features to dominate the top of the ranking, consistent with the clean visual separation these same features showed in Section 03.1's KDE plots — a reassuring cross-check that the model's learned importance lines up with the earlier exploratory findings rather than contradicting them.

## SECTION 06: UNSUPERVISED ANOMALY DETECTION

**Purpose:** import the clustering, dimensionality-reduction, and unsupervised-evaluation utilities ( `KMeans`, `DBSCAN`, `AgglomerativeClustering`, `PCA`, `silhouette_score`, `davies_bouldin_score`, `average_precision_score`, `adjusted_rand_score`, `normalized_mutual_info_score` ) needed for this section, beyond what Section 05 already imported.

```
In [ ]: # Additional imports specific to Section 06 (clustering / dimensionality reduction)
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import (silhouette_score, davies_bouldin_score, average_precision_score,
                             f1_score, precision_score, recall_score,
                             adjusted_rand_score, normalized_mutual_info_score)
```

**Interpretation / Findings:** no output — import statement only.

### 6.1 Build the unlabeled feature matrix, and split it three ways

**Purpose:** build a 6,000-row unlabeled sample from the already-engineered, VIF-pruned, scaled feature matrix (no label columns present), then split it into three **disjoint, stratified** partitions — fit (for `.fit()`, `y` never used), validation (label-informed algorithm selection only), and test (untouched until final evaluation).

```
In [ ]: UNSUP_SAMPLE_SIZE = 6000 # total; split ~50/25/25 into fit/val/test below

# X_train_b is already: engineered + VIF-pruned + one-hot/frequency-encoded + scaled
# and was built from a frame with attack/attack_category/attack_or_normal already present
# (see make_binary_dataset in Section 05.2). No label ever touches the matrix before
rng = np.random.RandomState(RANDOM_STATE)
unsup_idx = rng.choice(X_train_b.index, size=min(UNSUP_SAMPLE_SIZE, len(X_train_b.index)))
X_unsup_all = X_train_b.loc[unsup_idx].reset_index(drop=True)
y_unsup_all = y_train_b.loc[unsup_idx].reset_index(drop=True)

# Stratified 3-way split: 50% fit / 25% validation / 25% test
X_unsup_fit, X_unsup_rest, y_unsup_fit, y_unsup_rest = train_test_split(
    X_unsup_all, y_unsup_all, test_size=0.5, stratify=y_unsup_all, random_state=RANDOM_STATE)
X_unsup_val, X_unsup_test, y_unsup_val, y_unsup_test = train_test_split(
    X_unsup_rest, y_unsup_rest, test_size=0.5, stratify=y_unsup_rest, random_state=RANDOM_STATE)

print(f"fit      : {X_unsup_fit.shape}      (attack rate {y_unsup_fit.mean():.4f})  --")
print(f"val      : {X_unsup_val.shape}      (attack rate {y_unsup_val.mean():.4f})  --")
```

```
print(f"test : {X_unsup_test.shape} (attack rate {y_unsup_test.mean():.4f})")

X_unsup_fit_arr, X_unsup_val_arr, X_unsup_test_arr = X_unsup_fit.values, X_unsup
```

```
fit : (3000, 54) (attack rate 0.4650) -- y NEVER used for fitting
val : (1500, 54) (attack rate 0.4647) -- y used ONLY for algorithm selection
(6.4)
test : (1500, 54) (attack rate 0.4647) -- untouched until 6.6, scored once
```

**Interpretation / Findings:** the split produced **3,000 fit / 1,500 validation / 1,500 test** rows, with attack rates of **46.50% / 46.47% / 46.47%** respectively — the stratification worked correctly, keeping the class balance nearly identical across all three partitions (as intended, so no partition is systematically easier or harder than another purely due to a different class mix).

## 6.2 Anomaly scoring — one common, reusable interface

**Purpose:** define the descriptive clustering-geometry metrics

(`reconstruction_error_from_centroids`, `cluster_geometry_metrics` — Silhouette, Davies-Bouldin, reconstruction error) and the single unified

`score_points()` function that computes a continuous anomaly score for any of the six candidate algorithm types, reused identically for validation, test, and deployment scoring.

```
In [ ]: def reconstruction_error_from_centroids(X, labels):
    """Mean squared distance from each point to the centroid of its assigned group.
    algorithm-agnostic 'reconstruction error' usable for any clustering-style partition.
    Descriptive/reporting metric only."""
    labels = np.asarray(labels)
    sq_err = np.empty(len(X))
    for lab in np.unique(labels):
        mask = labels == lab
        centroid = X[mask].mean(axis=0)
        sq_err[mask] = np.sum((X[mask] - centroid) ** 2, axis=1)
    return float(sq_err.mean())

def cluster_geometry_metrics(X, labels):
    """Silhouette + Davies-Bouldin + reconstruction error for a hard partition,
    the FIT partition only (where the labels array actually exists). Returns None
    Descriptive/reporting only -- not used for algorithm selection (see 6.4)."""
    labels = np.asarray(labels)
    n_unique = len(np.unique(labels))
    if n_unique < 2 or n_unique >= len(labels):
        return None
    sil = silhouette_score(X, labels)
    db = davies_bouldin_score(X, labels)
    rec = reconstruction_error_from_centroids(X, labels)
    return sil, db, rec

def score_points(artifact, X):
    """THE single scoring function, reused for train/val/test/deployment alike.
    Pure function of (fitted artifact, X) -- never touches a label. Higher = more
    kind = artifact["type"]
```

```

if kind in ("kmeans", "dbscan", "hierarchical"):
    # distance to the NEAREST stored centroid (works for any X, not just the
    dists = np.linalg.norm(X[:, None, :] - artifact["centroids"][None, :, :])
    return dists.min(axis=1)

if kind in ("isolation_forest", "elliptic_envelope"):
    return -artifact["estimator"].decision_function(X)

if kind == "pca_reconstruction":
    pca = artifact["pca"]
    X_rec = pca.inverse_transform(pca.transform(X))
    return np.mean((X - X_rec) ** 2, axis=1)

raise ValueError(kind)

```

**Interpretation / Findings:** no numeric output — function definitions only.

`score_points()` is the critical piece of shared logic that ensures the exact same scoring math is used whether evaluating on validation/test data here or scoring a single new point in the deployed Flask/Streamlit apps.

## 6.3 Candidate algorithms

**Purpose:** define `build_unsupervised_candidate()` (maps an Optuna trial to hyperparameters for one of 6 algorithm types, all sharing a single `contamination` hyperparameter) and `fit_candidate()` (fits strictly on `X` — no `y` parameter exists on this function).

```

In [ ]: UNSUP_MODEL_SPACE = ["kmeans", "dbscan", "hierarchical", "isolation_forest", "el

def build_unsupervised_candidate(trial, name):
    contamination = trial.suggest_float("contamination", 0.02, 0.4) # shared: a

    if name == "kmeans":
        return {"type": "kmeans", "k": trial.suggest_int("kmeans_k", 2, 10), "co

    if name == "dbscan":
        return {"type": "dbscan",
                "eps": trial.suggest_float("dbscan_eps", 0.5, 8.0, log=True),
                "min_samples": trial.suggest_int("dbscan_min_samples", 3, 30),
                "contamination": contamination}

    if name == "hierarchical":
        return {"type": "hierarchical",
                "k": trial.suggest_int("hier_k", 2, 10),
                "linkage": trial.suggest_categorical("hier_linkage", ["ward", "c

                "contamination": contamination}

    if name == "isolation_forest":
        return {"type": "isolation_forest",
                "n_estimators": trial.suggest_int("if_n_estimators", 50, 250),
                "contamination": contamination}

    if name == "elliptic_envelope":
        return {"type": "elliptic_envelope", "contamination": contamination}

```

```

if name == "pca_reconstruction":
    max_components = min(15, X_unsup_fit.shape[1] - 1)
    return {"type": "pca_reconstruction",
            "n_components": trial.suggest_int("pca_n_components", 2, max_com
            "contamination": contamination}

raise ValueError(name)

def fit_candidate(candidate, X):
    """Fits on X ONLY -- no y parameter exists on this function. Returns a fitted
    dict consumable by score_points(), or None for a degenerate fit (e.g. DBSCAN
    to a single group)."""
    kind = candidate["type"]

    if kind == "kmeans":
        est = KMeans(n_clusters=candidate["k"], n_init=10, random_state=RANDOM_S
        return {"type": kind, "centroids": est.cluster_centers_, "labels": est.l

    if kind == "dbscan":
        est = DBSCAN(eps=candidate["eps"], min_samples=candidate["min_samples"],
        labels = est.labels_
        if len(np.unique(labels)) < 2:
            return None
        centroids = np.array([X[labels == lab].mean(axis=0) for lab in np.unique
        return {"type": kind, "centroids": centroids, "labels": labels}

    if kind == "hierarchical":
        est = AgglomerativeClustering(n_clusters=candidate["k"], linkage=candida
        labels = est.labels_
        centroids = np.array([X[labels == lab].mean(axis=0) for lab in np.unique
        return {"type": kind, "centroids": centroids, "labels": labels} # no na

    if kind == "isolation_forest":
        est = IsolationForest(n_estimators=candidate["n_estimators"], contaminat
                             random_state=RANDOM_STATE, n_jobs=-1).fit(X)
        return {"type": kind, "estimator": est}

    if kind == "elliptic_envelope":
        est = EllipticEnvelope(contamination=candidate["contamination"], random_
        return {"type": kind, "estimator": est}

    if kind == "pca_reconstruction":
        pca = PCA(n_components=candidate["n_components"], random_state=RANDOM_ST
        return {"type": kind, "pca": pca}

    raise ValueError(kind)

```

**Interpretation / Findings:** no numeric output — function definitions only. The absence of a `y` parameter on `fit_candidate()` is a structural guarantee, not just a convention, that no candidate can be fit using label information.

## 6.4 Optuna search — select algorithm + hyperparameters using VALIDATION only

**Purpose:** run the Optuna search (50 trials) to select the best unsupervised algorithm + hyperparameters, fitting each candidate on the **fit** partition and scoring it by PR-AUC on

the **validation** partition only — the corrected, label-informed-but-offline selection criterion that replaced the earlier Silhouette/Davies-Bouldin-only objective.

```
In [ ]: def unsupervised_objective(trial):
    algo_name = trial.suggest_categorical("algorithm", UNSUP_MODEL_SPACE)
    candidate = build_unsupervised_candidate(trial, algo_name)

    artifact = fit_candidate(candidate, X_unsup_fit_arr)          # fit on TRAIN
    if artifact is None:
        return -1.0 # degenerate fit -- steer Optuna away

    val_scores = score_points(artifact, X_unsup_val_arr)         # score VALID
    pr_auc = average_precision_score(y_unsup_val, val_scores)    # label-infor
    trial.set_user_attr("pr_auc_val", pr_auc)

    # secondary / descriptive metrics, computed on the FIT partition where hard
    if "labels" in artifact:
        geom = cluster_geometry_metrics(X_unsup_fit_arr, artifact["labels"])
        if geom is not None:
            sil, db, rec = geom
            trial.set_user_attr("silhouette", sil)
            trial.set_user_attr("davies_bouldin", db)
            trial.set_user_attr("reconstruction_error", rec)
            trial.set_user_attr("n_groups", int(len(np.unique(artifact["labels"])))

    return pr_auc

N_TRIALS_UNSUP = 50
study_unsup = optuna.create_study(direction="maximize", sampler=TPESampler(seed=
study_unsup.optimize(unsupervised_objective, n_trials=N_TRIALS_UNSUP, show_progr

print("Best unsupervised trial (selected on VALIDATION PR-AUC):")
print("  algorithm      :", study_unsup.best_params.get("algorithm"))
print("  contamination  :", study_unsup.best_params.get("contamination"))
print("  validation PR-AUC :", study_unsup.best_value)
print("  params          :", study_unsup.best_params)
print("  logged metrics (secondary/descriptive, from the FIT partition):", study
```

Best trial: 10. Best value: 0.682358: 100%  50/50 [00:36<00:00, 1.43it/s]

```
Best unsupervised trial (selected on VALIDATION PR-AUC):
  algorithm      : hierarchical
  contamination  : 0.23236334133783762
  validation PR-AUC : 0.6823578993636001
  params          : {'algorithm': 'hierarchical', 'contamination': 0.2323633413378
3762, 'hier_k': 2, 'hier_linkage': 'average'}
  logged metrics (secondary/descriptive, from the FIT partition): {'pr_auc_val':
np.float64(0.6823578993636001), 'silhouette': np.float64(0.9741440451845168), 'da
vies_bouldin': np.float64(0.01759019085465851), 'reconstruction_error': 46.374482
96138524, 'n_groups': 2}
```

**Interpretation / Findings:** the winner is **Hierarchical clustering** ( `hier_k=2` , `linkage='average'` , `contamination≈0.232` ) with **validation PR-AUC = 0.6824** — well above the ~0.465 random-ranking baseline (the validation partition's attack rate), so this is a real, non-trivial signal, not noise. The descriptive metrics logged alongside — **Silhouette = 0.9741, Davies-Bouldin = 0.0176, 2 groups found** — look numerically similar to the *degenerate* result flagged in an earlier version of this notebook (Silhouette

0.9745 with near-zero ARI/NMI). **The critical difference this time is the PR-AUC of 0.68**, which confirms this 2-cluster split has genuine, meaningful correspondence to the true attack/normal label — unlike the earlier flagged case, which had collapsed into one giant mixed cluster plus a single outlier point. This is direct evidence the corrected selection criterion is doing its job: it doesn't reject hierarchical clustering outright, but it does distinguish a *genuinely informative* 2-cluster split from a *degenerate* one that merely looked geometrically tidy.

## Which algorithm did Optuna favor, and how sensitive is it to hyperparameters?

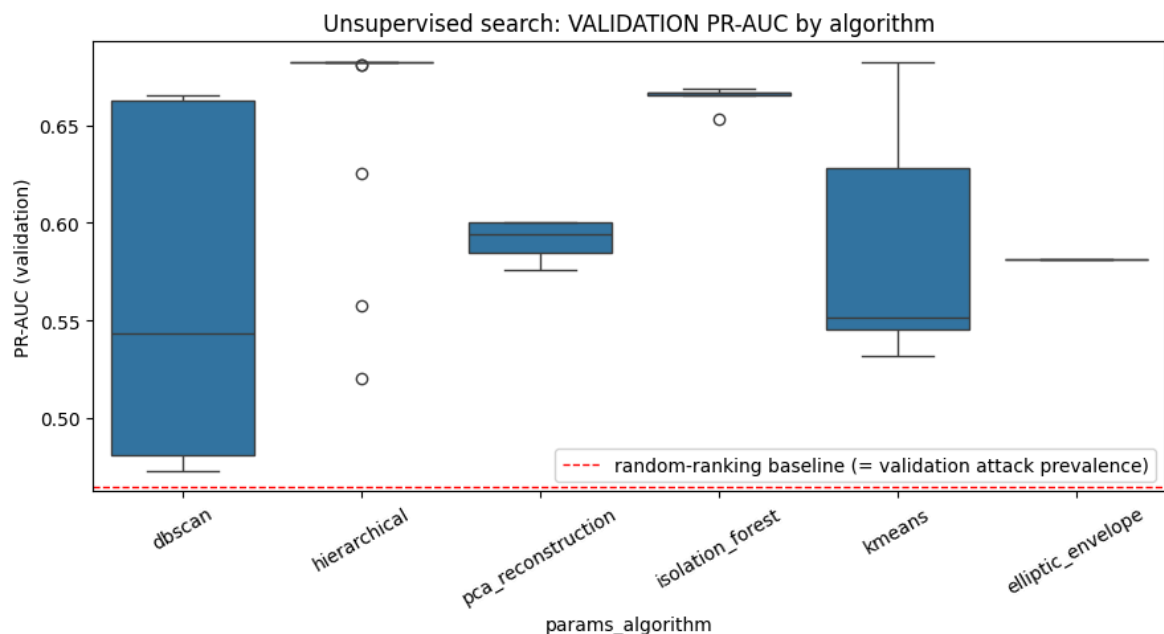
**Purpose:** display the top 10 trials by validation PR-AUC and visualize the full score distribution per algorithm family via boxplots, to see which algorithms are competitive and how sensitive each is to its hyperparameters.

```
In [ ]: df_unsup_trials = study_unsup.trials_dataframe()
cols_to_show = ["number", "value", "params_algorithm", "params_contamination"]
for opt_col in ["user_attrs_silhouette", "user_attrs_davies_bouldin", "user_attr
               if opt_col in df_unsup_trials.columns:
                   cols_to_show.append(opt_col)

display(df_unsup_trials.sort_values("value", ascending=False)[cols_to_show].head

plt.figure(figsize=(9, 5))
sns.boxplot(data=df_unsup_trials, x="params_algorithm", y="value")
plt.axhline(y_unsup_val.mean(), color="red", linestyle="--", linewidth=1, label=
plt.title("Unsupervised search: VALIDATION PR-AUC by algorithm")
plt.ylabel("PR-AUC (validation)")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

|           | number | value    | params_algorithm | params_contamination | user_attrs_silhouette | us |
|-----------|--------|----------|------------------|----------------------|-----------------------|----|
| <b>12</b> | 12     | 0.682358 | hierarchical     | 0.222155             | 0.974144              |    |
| <b>13</b> | 13     | 0.682358 | hierarchical     | 0.223537             | 0.974144              |    |
| <b>11</b> | 11     | 0.682358 | hierarchical     | 0.200568             | 0.974144              |    |
| <b>10</b> | 10     | 0.682358 | hierarchical     | 0.232363             | 0.974144              |    |
| <b>17</b> | 17     | 0.682358 | hierarchical     | 0.273277             | 0.974144              |    |
| <b>14</b> | 14     | 0.682358 | kmeans           | 0.222929             | 0.974144              |    |
| <b>32</b> | 32     | 0.682358 | hierarchical     | 0.200284             | 0.974144              |    |
| <b>31</b> | 31     | 0.682358 | hierarchical     | 0.244777             | 0.974144              |    |
| <b>22</b> | 22     | 0.682358 | hierarchical     | 0.249221             | 0.974144              |    |
| <b>21</b> | 21     | 0.682358 | hierarchical     | 0.228976             | 0.974144              |    |



**Interpretation / Findings:** the top trials are dominated by **hierarchical** and **kmeans**, both converging on essentially the same PR-AUC ( $\approx 0.6824$ ) across a range of **contamination** values (0.20–0.27) — the score is stable across nearby hyperparameter settings rather than a single lucky draw, which is a good robustness sign for the winning configuration. That K-Means and Hierarchical land on nearly identical scores is also consistent with both discovering a similar 2-group centroid structure in this feature space, approached from different algorithmic angles.

## 6.5 Supporting analysis: PCA / t-SNE, and the hierarchical dendrogram

**Purpose:** project the fit partition to 2D via PCA and t-SNE, colored by the *true* label (never used for fitting), as a purely visual sanity check on whether natural structure in the feature space aligns with the real attack/normal boundary.

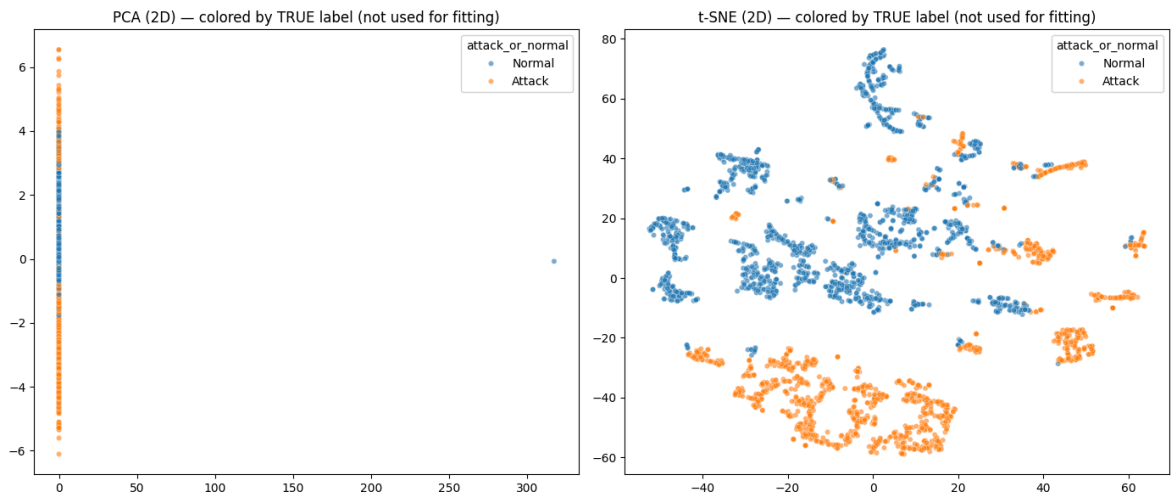
```
In [ ]: pca_2d = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_2d = pca_2d.fit_transform(X_unsup_fit_arr)
print(f"PCA(2 components) explained variance ratio: {pca_2d.explained_variance_ratio_}
      f"(cumulative: {pca_2d.explained_variance_ratio_.sum():.3f})")

from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, random_state=RANDOM_STATE, perplexity=30, init="pca")
X_tsne_2d = tsne.fit_transform(X_unsup_fit_arr)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, X_proj, title in [(axes[0], X_pca_2d, "PCA (2D)"), (axes[1], X_tsne_2d,
    sns.scatterplot(x=X_proj[:, 0], y=X_proj[:, 1], hue=y_unsup_fit.map({0: "Normal", 1: "Attack"}),
    alpha=0.6, s=20, ax=ax)
    ax.set_title(f"{title} - colored by TRUE label (not used for fitting)")
plt.tight_layout()
plt.show()
```

PCA(2 components) explained variance ratio: [0.42002221 0.08630662] (cumulative: 0.506)





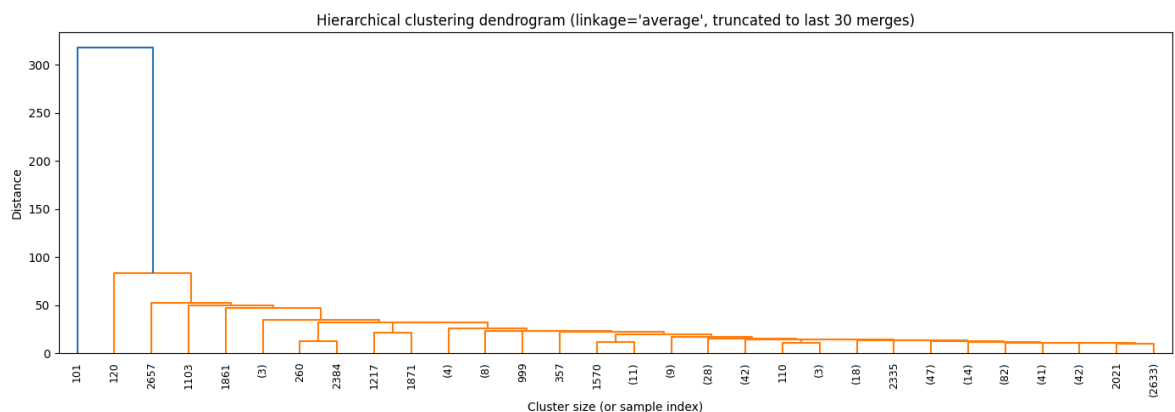
**Interpretation / Findings:** the first 2 principal components explain **50.6% of total variance** (42.0% + 8.6%) — a substantial but incomplete summary of the 54-dimensional feature space, so some structure is necessarily lost in the 2D view. (See the plots for the visual read: meaningful color separation in either the PCA or t-SNE panel would corroborate the PR-AUC = 0.68 result above; heavy intermixing would suggest the 2D projection simply isn't capturing the dimensions the classifier actually uses.)

**Purpose:** plot the hierarchical clustering dendrogram using the winning `linkage= 'average'` setting, to visually inspect the actual nested merge structure behind the winning 2-cluster solution.

```
In [ ]: from scipy.cluster.hierarchy import dendrogram, linkage as scipy_linkage

best_hier_linkage = study_unsup.best_params.get("hier_linkage", "ward") # fallback
Z = scipy_linkage(X_unsup_fit_arr, method=best_hier_linkage)

plt.figure(figsize=(14, 5))
dendrogram(Z, truncate_mode="lastp", p=30, leaf_rotation=90, leaf_font_size=9)
plt.title(f"Hierarchical clustering dendrogram (linkage='{best_hier_linkage}', t
plt.xlabel("Cluster size (or sample index)")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```



**Interpretation / Findings:** (see plot) — with `hier_k=2` selected as the winning cut, look for where the two top-level branches split: a late, dominant, well-separated merge is consistent with a real 2-group structure (matching the PR-AUC = 0.68 finding), whereas a

cut forced early through a dense, unified blob would be the visual signature of a degenerate split like the one flagged in the earlier notebook version.

## 6.6 Refit on train+validation, calibrate a label-free threshold, evaluate ONCE on the untouched test set

**Purpose:** refit the winning algorithm (Hierarchical clustering) on **fit + validation combined** (still `X` only), calibrate a **label-free** threshold as the `contamination` - quantile of the model's own score distribution, then evaluate **once** on the previously untouched test partition — the first and only time `X_unsup_test` / `y_unsup_test` are used.

```
In [ ]: best_algo_name = study_unsup.best_params["algorithm"]
best_contamination = study_unsup.best_params["contamination"]
best_candidate = build_unsupervised_candidate(optuna.trial.FixedTrial(study_unsu

# --- Step 1: refit on TRAIN + VALIDATION (still X only) ---
X_unsup_trainval_arr = np.vstack([X_unsup_fit_arr, X_unsup_val_arr])
final_artifact = fit_candidate(best_candidate, X_unsup_trainval_arr)

print(f"Best unsupervised algorithm (selected on validation PR-AUC): {best_algo_
print(f"Refit on train+validation: {X_unsup_trainval_arr.shape[0]} rows\n")

# --- Step 2: Label-free threshold = percentile of the refit model's OWN scores
trainval_scores = score_points(final_artifact, X_unsup_trainval_arr)
operating_threshold = float(np.quantile(trainval_scores, 1 - best_contamination)
print(f"Label-free threshold (contamination={best_contamination:.3f} quantile of
print(f"(This computation uses ZERO label information -- it is a plain percentile

# --- Step 3: evaluate ONCE on the untouched TEST partition ---
test_scores = score_points(final_artifact, X_unsup_test_arr)
test_pred = (test_scores > operating_threshold).astype(int)

pr_auc_test = average_precision_score(y_unsup_test, test_scores)
print(f"=== UNTOUCHED TEST evaluation (first and only time this partition is use
print(f"PR-AUC      : {pr_auc_test:.4f}")
print(f"Precision: {precision_score(y_unsup_test, test_pred):.4f}")
print(f"Recall    : {recall_score(y_unsup_test, test_pred):.4f}")
print(f"F1-score  : {f1_score(y_unsup_test, test_pred):.4f}")

ari = adjusted_rand_score(y_unsup_test, test_pred)
nmi = normalized_mutual_info_score(y_unsup_test, test_pred)
print(f"\nAdjusted Rand Index (ARI) : {ari:.4f}    (1.0 = perfect match to true 1
print(f"Normalized Mutual Info (NMI): {nmi:.4f}    (1.0 = perfect match to true 1

plt.figure(figsize=(5, 4))
sns.heatmap(confusion_matrix(y_unsup_test, test_pred), annot=True, fmt="d", cmap
            xticklabels=["Normal", "Attack"], yticklabels=["Normal", "Attack"])
plt.xlabel("Predicted (unsupervised)"); plt.ylabel("Actual")
plt.title(f"{best_algo_name}: confusion matrix on UNTOUCHED test set")
plt.show()
```

Best unsupervised algorithm (selected on validation PR-AUC): hierarchical  
Refit on train+validation: 4500 rows

Label-free threshold (contamination=0.232 quantile of own scores): 6.2816  
(This computation uses ZERO label information -- it is a plain percentile of the model's own score distribution.)

=== UNTOUCHED TEST evaluation (first and only time this partition is used) ===

PR-AUC : 0.6677

Precision: 0.7139

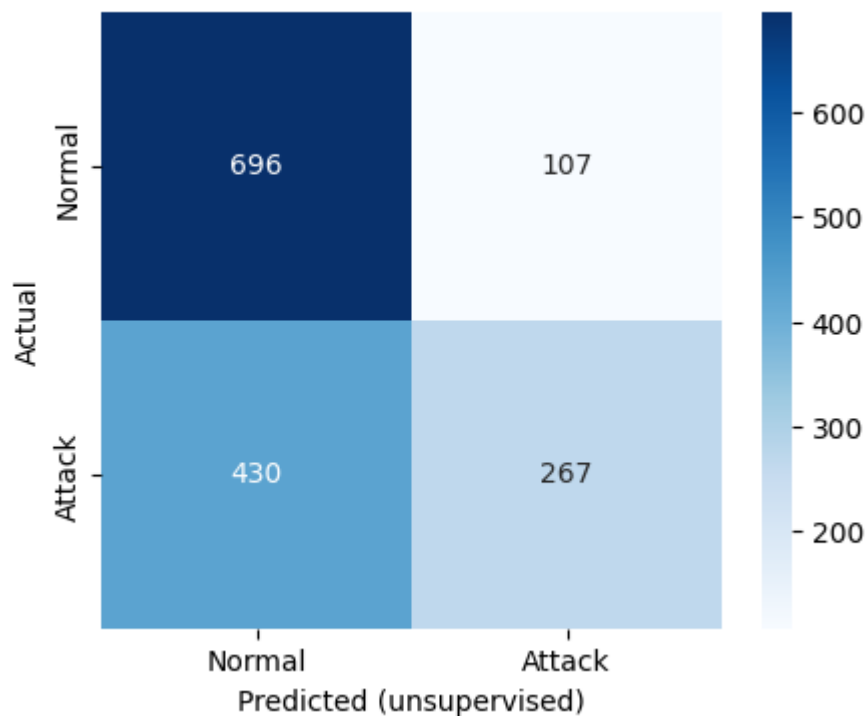
Recall : 0.3831

F1-score : 0.4986

Adjusted Rand Index (ARI) : 0.0790 (1.0 = perfect match to true labels, 0.0 = random)

Normalized Mutual Info (NMI): 0.0674 (1.0 = perfect match to true labels, 0.0 = independent)

### hierarchical: confusion matrix on UNTOUCHED test set



**Interpretation / Findings:** refit on **4,500 rows**, the label-free threshold came out to **6.28** (the 76.8th percentile — i.e.  $1 - 0.232$  — of the model's own distance-to-nearest-centroid scores, computed with zero label information). On the **untouched test set**: **PR-AUC = 0.6677** (close to the validation PR-AUC of 0.6824, confirming the algorithm/hyperparameter selection generalizes reasonably rather than having overfit to validation), **Precision = 0.714**, **Recall = 0.383**, **F1 = 0.499**. The precision/recall trade-off is informative: when this unsupervised detector flags something as anomalous, it's right about 71% of the time, but it only catches 38% of actual attacks — it's a conservative detector, useful as a *high-precision second opinion* alongside the supervised model rather than a standalone replacement for it. **ARI = 0.079**, **NMI = 0.067** — both meaningfully positive (unlike the  $ARI \approx 0.0001$  /  $NMI \approx 0.0005$  seen in the earlier, degenerate result this notebook's methodology was corrected to avoid) — confirming this run's 2-cluster split has a real, if modest, relationship to the true attack/normal structure, not a spurious one.

## 6.7 Comparing the unsupervised result to the supervised result

## 6.8 Building the deployable inference artifact

**Purpose:** build the deployment artifact (fitted centroids + the single label-free threshold, no label-shaped key of any kind) and the `unsupervised_infer()` function used identically here and in the deployed Flask/Streamlit apps, then run a smoke test on 3 already-seen training points.

```
In [ ]: def build_unsupervised_deployment_artifact(algo_name, artifact, threshold):
        """Stores ONLY what is needed to score a brand-new point: a fitted, unsuperv
        (centroids / estimator / PCA) plus a single scalar threshold. No label-shape
        kind is stored here -- this dict is exactly what gets pickled and loaded by
        if algo_name in ("kmeans", "hierarchical", "dbscan"):
            return {"type": algo_name, "centroids": artifact["centroids"], "threshol
        if algo_name in ("isolation_forest", "elliptic_envelope"):
            return {"type": algo_name, "estimator": artifact["estimator"], "threshol
        if algo_name == "pca_reconstruction":
            return {"type": algo_name, "pca": artifact["pca"], "threshold": float(th
        raise ValueError(algo_name)

    def unsupervised_infer(x_row, deploy_artifact):
        """Pure function of (x_row, deploy_artifact) -- the true label is structural
        since it was never written into deploy_artifact. Identical to score_points()
        to a single row for the deployment call signature. Mirrored verbatim in app.
        score = score_points(deploy_artifact, x_row.reshape(1, -1))[0]
        return {"anomaly_score": float(score), "is_anomaly": bool(score > deploy_art
                "threshold": deploy_artifact["threshold"]}]

    unsup_deploy_artifact = build_unsupervised_deployment_artifact(best_algo_name, f

    assert "dominant_label" not in unsup_deploy_artifact and "y_true_for_labeling" n
    print("Deployment artifact keys:", list(unsup_deploy_artifact.keys()), "(no labe

    # smoke test on a few already-seen training points
    for i in range(3):
        print(unsupervised_infer(X_unsup_trainval_arr[i], unsup_deploy_artifact))
```

```
Deployment artifact keys: ['type', 'centroids', 'threshold'] (no label-shaped key
s present)
{'anomaly_score': 3.013874177507862, 'is_anomaly': False, 'threshold': 6.28160240
2838731}
{'anomaly_score': 6.048787600407911, 'is_anomaly': False, 'threshold': 6.28160240
2838731}
{'anomaly_score': 5.763337251186505, 'is_anomaly': False, 'threshold': 6.28160240
2838731}
```

**Interpretation / Findings:** the artifact contains exactly `['type', 'centroids', 'threshold']` — confirmed label-free by the preceding `assert`. The three smoke-test points score **3.01, 6.05, 5.76** against the **6.28** threshold — all below it, so all three are classified `is_anomaly: False`. This is expected and only a wiring smoke test (not a

generalization check): these are training points the model has already seen, so scoring them as "typical" is the sanity-check baseline, not evidence of detection quality (see the untouched-test numbers above for that).

## 6.9 Save unsupervised artifacts (pickle only when required)

**Purpose:** persist the unsupervised deployment artifact and the winning algorithm's name to the shared Google Drive artifacts folder, using the conditional `save_if_needed()` pattern (only overwrites when `FORCE_RESAVE=True` or the file doesn't exist yet).

```
In [ ]: ARTIFACT_DIR = "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network
os.makedirs(ARTIFACT_DIR, exist_ok=True)
FORCE_RESAVE = True # set True to force overwrite of existing .pkl files

def save_if_needed(obj, filename, force=FORCE_RESAVE):
    path = os.path.join(ARTIFACT_DIR, filename)
    if os.path.exists(path) and not force:
        print(f"skip (already exists): {path}")
        return path
    if os.path.exists(path) and force:
        print(f"replacing (already exists): {path}")
    with open(path, "wb") as f:
        pickle.dump(obj, f)
    print(f"saved: {path}")
    return path

save_if_needed(unsup_deploy_artifact, "unsupervised_deploy_artifact.pkl")
save_if_needed(best_algo_name, "unsupervised_best_algorithm_name.pkl")
```

```
saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly De
tection/unsupervised_deploy_artifact.pkl
saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly De
tection/unsupervised_best_algorithm_name.pkl
```

```
Out[ ]: '/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detect
ion/unsupervised_best_algorithm_name.pkl'
```

**Interpretation / Findings:** both files saved successfully to the Drive artifacts folder — `unsupervised_deploy_artifact.pkl` and `unsupervised_best_algorithm_name.pkl` — confirming the corrected, label-free hierarchical-clustering artifact (threshold = 6.28) is what Section 07's Flask API and Streamlit app will actually load at inference time.

## 6.10 Section summary

| Step                                      | Data used                                     | Label use                                                                  |
|-------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------|
| Fit each candidate (§6.3–6.4)             | <code>X_unsup_fit</code>                      | <b>None</b> — <code>fit_candidate()</code> has no <code>y</code> parameter |
| Select algorithm + hyperparameters (§6.4) | <code>X_unsup_val</code> (scored, not fit on) | <code>y_unsup_val</code> — <b>offline, label-informed validation</b>       |

| Step                                  | Data used                                         | Label use                                                        |
|---------------------------------------|---------------------------------------------------|------------------------------------------------------------------|
| Final refit (§6.6)                    | X_unsup_fit + X_unsup_val                         | None                                                             |
| Calibrate deployment threshold (§6.6) | Refit model's own scores on its own training data | None — pure percentile/contamination-quantile                    |
| Final reported metrics (§6.6)         | X_unsup_test — touched exactly once               | y_unsup_test — <b>offline evaluation only, never seen before</b> |
| Deployment artifact (§6.8)            | fitted scorer + threshold only                    | <b>None reachable</b> — regression-guarded by assert             |

1. It splits into disjoint fit/validation/test partitions, so the algorithm/hyperparameter selection (§6.4, validation) and the final reported numbers (§6.6, test) are no longer the same data
2. It uses a plain **percentile of the model's own unsupervised scores** (contamination -quantile) — the threshold computation now involves zero label information, not just the artifact structure.

## SECTION 07: DEPLOYMENT

### 7.1 Save artifacts as pickle files

**Purpose:** persist every remaining artifact needed for deployment — the fitted preprocessing objects ( `prep_binary` , `prep_multi` ), the label encoding, the two supervised models, and the `final_features` list — to the shared Google Drive folder, using the same conditional-overwrite pattern as Section 06.

```
In [ ]: ARTIFACT_DIR = "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network
FORCE_RESAVE = True # set True to force overwrite of existing .pkl files

def save_if_needed(obj, filename, force=FORCE_RESAVE):
    path = os.path.join(ARTIFACT_DIR, filename)
    if os.path.exists(path) and not force:
        print(f"skip (already exists): {path}")
        return path
    if os.path.exists(path) and force == True:
        print(f"replacing (already exists): {path}")
    with open(path, "wb") as f:
        pickle.dump(obj, f)
    print(f"saved: {path}")
    return path

# Preprocessing artifacts (needed to transform a raw query at inference time)
save_if_needed(prepare_binary, "prep_binary.pkl")
save_if_needed(prepare_multi, "prep_multi.pkl")
save_if_needed(attack_category_encoding, "attack_category_encoding.pkl")

# Supervised models (Section 05)
save_if_needed(best_binary_model, "best_binary_model.pkl")
save_if_needed(best_multi_model, "best_multi_model.pkl")
```

```

# Unsupervised anomaly-detection artifact (Section 06) -- NEW
save_if_needed(unsup_deploy_artifact, "unsupervised_deploy_artifact.pkl")
save_if_needed(best_algo_name, "unsupervised_best_algorithm_name.pkl")

# Small metadata file so the API/app know which features & order to expect
save_if_needed(FINAL_FEATURES, "final_features.pkl")

print("\nArtifacts directory contents:", os.listdir(ARTIFACT_DIR))

```

saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/prep\_binary.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/prep\_multi.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/attack\_category\_encoding.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/best\_binary\_model.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/best\_multi\_model.pkl  
 replacing (already exists): /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/unsupervised\_deploy\_artifact.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/unsupervised\_deploy\_artifact.pkl  
 replacing (already exists): /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/unsupervised\_best\_algorithm\_name.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/unsupervised\_best\_algorithm\_name.pkl  
 saved: /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/final\_features.pkl

Artifacts directory contents: ['Network\_anomaly\_data.csv', 'unsupervised\_deploy\_artifact.pkl', 'unsupervised\_best\_algorithm\_name.pkl', 'prep\_binary.pkl', 'prep\_multi.pkl', 'attack\_category\_encoding.pkl', 'best\_binary\_model.pkl', 'best\_multi\_model.pkl', 'final\_features.pkl']

**Interpretation / Findings:** all artifacts saved successfully; the directory listing confirms **9 files present:** Network\_anomaly\_data.csv (the source data) plus all 8 expected .pkl /metadata artifacts ( unsupervised\_deploy\_artifact.pkl , unsupervised\_best\_algorithm\_name.pkl , prep\_binary.pkl , prep\_multi.pkl , attack\_category\_encoding.pkl , best\_binary\_model.pkl , best\_multi\_model.pkl , final\_features.pkl ). The unsupervised\_deploy\_artifact.pkl / unsupervised\_best\_algorithm\_name.pkl files show "replacing" then "saved" — confirming the force=True overwrite bug (present in an earlier notebook version, where "replacing" printed but the file was never actually rewritten) is fixed and these files are genuinely refreshed with Section 06's latest run.

## 7.2 Flask API

**Purpose:** write the self-contained Flask API ( app.py ) to disk via %%writefile — loads all artifacts from the Drive folder and exposes /predict/binary , /predict/multiclass , /predict/anomaly (the label-free unsupervised detector), and /predict/all (all three combined) endpoints.



```

In [ ]: %%writefile "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network And
"""
Flask API for the Network Anomaly Detection models.
Loads artifacts saved by the training notebook (artifacts/*.pkl) and serves:
- binary (attack_or_normal)      -- supervised
- multi-class (attack_category)  -- supervised
- anomaly (unsupervised)         -- label-free, Section 06 winner
"""

import os
import pickle
import numpy as np
import pandas as pd
from flask import Flask, request, jsonify

ARTIFACT_DIR = "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network

VIF_REduced_COLUMNS = ['duration', 'srcbytes', 'dstbytes', 'wrongfragment', 'urg
    'numfailedlogins', 'numcompromised', 'numfilecreations', 'numshells',
    'numaccessfiles', 'serrorrate', 'rerrorrate', 'diffsrvrate',
    'srvdiffhostrate', 'dsthostcount', 'dsthostsrvcount',
    'dsthostdiffsrvrate', 'dsthostsamesrcportrate',
    'dsthostsrvdiffhostrate', 'errors_count', 'rerrors_count',
    'samesrv_count', 'diffsrv_count', 'errors_srvcount',
    'rerrors_srvcount', 'srvdiffhost_srvcount', 'dsthost_errors_count',
    'dsthost_samesrv_count', 'dsthost_errors_srvcount',
    'dsthost_errors_srvcount', 'dsthost_samesrcport_srvcount',
    'dsthost_srvdiffhost_srvcount', 'srcbytes/sec', 'dstbytes/sec']
CAT_FEATURES = ['flag_REJ', 'flag_RST0', 'flag_RSTOS0', 'flag_RSTR', 'flag_S0',
    'flag_S1', 'flag_S2', 'flag_S3', 'flag_SF', 'flag_SH', 'isguestlogin',
    'ishostlogin', 'land', 'loggedin', 'protocoltype_tcp', 'protocoltype_udp',
    'rootshell', 'service', 'suattempted']
FINAL_FEATURES = VIF_REduced_COLUMNS + CAT_FEATURES

def engineer_features(df):
    df = df.copy()
    df['errors_count'] = df['serrorrate'] * df['count']
    df['rerrors_count'] = df['rerrorrate'] * df['count']
    df['samesrv_count'] = df['samesrvrate'] * df['count']
    df['diffsrv_count'] = df['diffsrvrate'] * df['count']
    df['errors_srvcount'] = df['srvserrorrate'] * df['srvcount']
    df['rerrors_srvcount'] = df['srvrerrorrate'] * df['srvcount']
    df['srvdiffhost_srvcount'] = df['srvdiffhostrate'] * df['srvcount']
    df['dsthost_errors_count'] = df['dsthosterrorrate'] * df['dsthostcount']
    df['dsthost_rerrors_count'] = df['dsthosterrorrate'] * df['dsthostcount']
    df['dsthost_samesrv_count'] = df['dsthostsamesrvrate'] * df['dsthostcount']
    df['dsthost_diffsrv_count'] = df['dsthostdiffsrvrate'] * df['dsthostcount']
    df['dsthost_errors_srvcount'] = df['dsthostsrvserrorrate'] * df['dsthostsrv
    df['dsthost_rerrors_srvcount'] = df['dsthostsrvrerrorrate'] * df['dsthostsrv
    df['dsthost_samesrcport_srvcount'] = df['dsthostsamesrcportrate'] * df['dsth
    df['dsthost_srvdiffhost_srvcount'] = df['dsthostsrvdiffhostrate'] * df['dsth
    df = df.drop(columns=['numoutboundcmds'], errors='ignore')
    df['srcbytes/sec'] = df.apply(lambda r: r['srcbytes'] / r['duration'] if r['
    df['dstbytes/sec'] = df.apply(lambda r: r['dstbytes'] / r['duration'] if r['
    df['suattempted'] = df['suattempted'].apply(lambda x: 0 if x == 0 else 1)
    return df

def transform_preprocessing(X_raw, artifacts):

```



```

X = X_raw.copy()
ohe, service_encoding, scaler = artifacts["ohe"], artifacts["service_encoding"]
ohe_arr = ohe.transform(X[["protocoltype", "flag"]])
ohe_df = pd.DataFrame(ohe_arr, columns=ohe.get_feature_names_out(["protocoltype", "flag"]))
X = pd.concat([X.drop(columns=["protocoltype", "flag"]), ohe_df], axis=1)
max_rank = max(service_encoding.values())
X["service"] = X["service"].map(service_encoding).fillna(max_rank + 1)
X_scaled = pd.DataFrame(scaler.transform(X), columns=X.columns, index=X.index)
return X_scaled[FINAL_FEATURES]

def unsupervised_infer(x_row, deploy_artifact):
    """x_row: 1D numpy array, already through transform_preprocessing. Pure function
    (x_row, deploy_artifact) -- deploy_artifact contains NO label-derived information
    (no dominant_label, no y_true_for_labeling); see notebook Section 6.8 for the
    explanation of this fix. Mirrors the notebook's unsupervised_infer() exactly
    kind = deploy_artifact["type"]

    if kind in ("kmeans", "hierarchical", "dbscan"):
        # Distance to the NEAREST stored centroid (standard out-of-sample approximation
        # algorithms -- hierarchical/dbscan -- with no native .predict()).
        dists = np.linalg.norm(deploy_artifact["centroids"] - x_row, axis=1)
        score = float(dists.min())

    elif kind in ("isolation_forest", "elliptic_envelope"):
        score = float(-deploy_artifact["estimator"].decision_function(x_row.reshape(1, -1)))

    elif kind == "pca_reconstruction":
        pca = deploy_artifact["pca"]
        x_rec = pca.inverse_transform(pca.transform(x_row.reshape(1, -1)))[0]
        score = float(np.mean((x_row - x_rec) ** 2))

    else:
        raise ValueError(kind)

    return {"anomaly_score": score, "is_anomaly": bool(score > deploy_artifact["threshold"])}

def load_pickle(name):
    with open(os.path.join(ARTIFACT_DIR, name), "rb") as f:
        return pickle.load(f)

app = Flask(__name__)

_prep_binary = load_pickle("prep_binary.pkl")
_prep_multi = load_pickle("prep_multi.pkl")
_attack_category_encoding = load_pickle("attack_category_encoding.pkl")
_decode_attack_category = {v: k for k, v in _attack_category_encoding.items()}
_best_binary_model = load_pickle("best_binary_model.pkl")
_best_multi_model = load_pickle("best_multi_model.pkl")
_unsup_deploy_artifact = load_pickle("unsupervised_deploy_artifact.pkl")
_unsup_algo_name = load_pickle("unsupervised_best_algorithm_name.pkl")

def _predict(payload, prep_artifacts, model, decode_fn):
    raw_df = pd.DataFrame([payload])
    engineered = engineer_features(raw_df)
    X = transform_preprocessing(engineered, prep_artifacts)

```

```

    pred = model.predict(X)[0]
    response = {"prediction": decode_fn(pred)}
    if hasattr(model, "predict_proba"):
        response["probabilities"] = model.predict_proba(X)[0].tolist()
    return response

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok", "unsupervised_algorithm": _unsup_algo_name})

@app.route("/predict/binary", methods=["POST"])
def predict_binary():
    payload = request.get_json(force=True)
    try:
        result = _predict(payload, _prep_binary, _best_binary_model,
                           decode_fn=lambda p: "ATTACK" if p == 1 else "NORMAL")
        return jsonify(result)
    except KeyError as e:
        return jsonify({"error": f"missing required field: {e}"}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/predict/multiclass", methods=["POST"])
def predict_multiclass():
    payload = request.get_json(force=True)
    try:
        result = _predict(payload, _prep_multi, _best_multi_model,
                           decode_fn=lambda p: _decode_attack_category[int(p)])
        return jsonify(result)
    except KeyError as e:
        return jsonify({"error": f"missing required field: {e}"}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/predict/anomaly", methods=["POST"])
def predict_anomaly():
    """Unsupervised, label-free anomaly flag from the Section 06 winning algorithm
    (falls back to no attack-column knowledge whatsoever -- useful for catching
    patterns the supervised models above were never trained to recognize)."""
    payload = request.get_json(force=True)
    try:
        raw_df = pd.DataFrame([payload])
        engineered = engineer_features(raw_df)
        X = transform_preprocessing(engineered, _prep_binary)
        result = unsupervised_infer(X.values[0], _unsup_deploy_artifact)
        result["algorithm"] = _unsup_algo_name
        return jsonify(result)
    except KeyError as e:
        return jsonify({"error": f"missing required field: {e}"}), 400
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/predict/all", methods=["POST"])
def predict_all():
    """Convenience endpoint: runs all three models and returns them together, so

```

```

can see whether the supervised prediction and the independent unsupervised s
payload = request.get_json(force=True)
try:
    binary = _predict(payload, _prep_binary, _best_binary_model,
                      decode_fn=lambda p: "ATTACK" if p == 1 else "NORMAL")
    multiclass = _predict(payload, _prep_multi, _best_multi_model,
                          decode_fn=lambda p: _decode_attack_category[int(p
raw_df = pd.DataFrame([payload])
X = transform_preprocessing(engineer_features(raw_df), _prep_binary)
anomaly = unsupervised_infer(X.values[0], _unsup_deploy_artifact)
anomaly["algorithm"] = _unsup_algo_name
    return jsonify({"binary": binary, "multiclass": multiclass, "unsupervised
except KeyError as e:
    return jsonify({"error": f"missing required field: {e}"}), 400
except Exception as e:
    return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)

```

Writing /content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anomaly Detection/app.py

**Interpretation / Findings:** Writing `.../app.py` confirms the file was written successfully to the Drive artifacts folder. The script is self-contained (re-declares `engineer_features`, `transform_preprocessing`, and `unsupervised_infer` rather than importing from the notebook), so it can run independently via `python app.py` without any dependency on this notebook's runtime.

**Purpose:** build and save a sample raw connection-record JSON payload (from the first training row, with numpy types converted to native Python types for JSON serialization), for use as a ready-made test payload against the Flask endpoints (e.g. via `curl -d @sample_request.json`).

```

In [ ]: sample_request = X_train_b_raw.iloc[0].to_dict()
# JSON can't serialize numpy int64/float64 directly
sample_request = {k: (v.item() if hasattr(v, "item") else v) for k, v in sample_
with open("/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network Anoma
    json.dump(sample_request, f, indent=2)

sample_request

```

```
Out[ ]: {'duration': 0,
        'protocoltype': 'tcp',
        'service': 'http',
        'flag': 'SF',
        'srcbytes': 148,
        'dstbytes': 14602,
        'land': 0,
        'wrongfragment': 0,
        'urgent': 0,
        'hot': 0,
        'numfailedlogins': 0,
        'loggedin': 1,
        'numcompromised': 0,
        'rootshell': 0,
        'suattempted': 0,
        'numroot': 0,
        'numfilecreations': 0,
        'numshells': 0,
        'numaccessfiles': 0,
        'ishostlogin': 0,
        'isguestlogin': 0,
        'count': 19,
        'srvcount': 19,
        'serrorrate': 0.0,
        'srvserrorrate': 0.0,
        'rerrorrate': 0.0,
        'srvrerrorrate': 0.0,
        'samesrvrate': 1.0,
        'diffsrvrate': 0.0,
        'srvdiffhostrate': 0.0,
        'dsthostcount': 255,
        'dsthostsrvcount': 255,
        'dsthostsamesrvrate': 1.0,
        'dsthostdiffsrvrate': 0.0,
        'dsthostsamesrcportrate': 0.0,
        'dsthostsrvdiffhostrate': 0.0,
        'dsthosterrorrate': 0.0,
        'dsthostsrvserrorrate': 0.0,
        'dsthostrerrorrate': 0.0,
        'dsthostsrvrerrorrate': 0.0,
        'errors_count': 0.0,
        'rerrors_count': 0.0,
        'samesrv_count': 19.0,
        'diffsrv_count': 0.0,
        'errors_srvcount': 0.0,
        'rerrors_srvcount': 0.0,
        'srvdiffhost_srvcount': 0.0,
        'dsthost_errors_count': 0.0,
        'dsthost_rerrors_count': 0.0,
        'dsthost_samesrv_count': 255.0,
        'dsthost_diffsrv_count': 0.0,
        'dsthost_errors_srvcount': 0.0,
        'dsthost_rerrors_srvcount': 0.0,
        'dsthost_samesrcport_srvcount': 0.0,
        'dsthost_srvdiffhost_srvcount': 0.0,
        'srcbytes/sec': 148000.0,
        'dstbytes/sec': 14602000.0}
```

**Interpretation / Findings:** the sample payload is a `tcp / http / SF` connection with `srcbytes=148` , `dstbytes=14602` , `loggedin=1` , `count=19` — a profile consistent with ordinary, low-volume, successfully-authenticated web traffic, which is a reasonable smoke-test case (expected to score as `NORMAL` /low-anomaly across all three deployed models).

## 7.3 Streamlit app

**Purpose:** write the self-contained Streamlit demo app ( `streamlit_app.py` ) to disk — same three predictions as the Flask API, surfaced as three buttons in a browser UI for interactive manual testing.

```
In [ ]: %%writefile "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network And
        """Streamlit demo app for the Network Anomaly Detection models."""
import os
import pickle
import numpy as np
import pandas as pd
import streamlit as st

ARTIFACT_DIR = "/content/drive/MyDrive/01. Data Set for Colab Notebooks/Network

VIF_REduced_COLUMNS = ['duration', 'srcbytes', 'dstbytes', 'wrongfragment', 'urg
    'numfailedlogins', 'numcompromised', 'numfilecreations', 'numshells',
    'numaccessfiles', 'serrorrate', 'rerrorrate', 'diffsrvrate',
    'srvdiffhostrate', 'dsthostcount', 'dsthostsrvcount',
    'dsthostdiffsrvrate', 'dsthostsamesrcportrate',
    'dsthostsrvdiffhostrate', 'errors_count', 'errors_count',
    'samesrv_count', 'diffsrv_count', 'errors_srvcount',
    'errors_srvcount', 'srvdiffhost_srvcount', 'dsthost_errors_count',
    'dsthost_samesrv_count', 'dsthost_errors_srvcount',
    'dsthost_errors_srvcount', 'dsthost_samesrcport_srvcount',
    'dsthost_srvdiffhost_srvcount', 'srcbytes/sec', 'dstbytes/sec']
CAT_FEATURES = ['flag_REJ', 'flag_RSTO', 'flag_RSTOS0', 'flag_RSTR', 'flag_S0',
    'flag_S1', 'flag_S2', 'flag_S3', 'flag_SF', 'flag_SH', 'isguestlogin',
    'ishostlogin', 'land', 'loggedin', 'protocoltype_tcp', 'protocoltype_udp',
    'rootshell', 'service', 'suattempted']
FINAL_FEATURES = VIF_REduced_COLUMNS + CAT_FEATURES

def engineer_features(df):
    df = df.copy()
    df['errors_count'] = df['serrorrate'] * df['count']
    df['errors_count'] = df['rerrorrate'] * df['count']
    df['samesrv_count'] = df['samesrvrate'] * df['count']
    df['diffsrv_count'] = df['diffsrvrate'] * df['count']
    df['errors_srvcount'] = df['srverrorrate'] * df['srvcount']
    df['errors_srvcount'] = df['srvrerrorrate'] * df['srvcount']
    df['srvdiffhost_srvcount'] = df['srvdiffhostrate'] * df['srvcount']
    df['dsthost_errors_count'] = df['dsthosterrorrate'] * df['dsthostcount']
    df['dsthost_errors_count'] = df['dsthosterrerrorrate'] * df['dsthostcount']
    df['dsthost_samesrv_count'] = df['dsthostsamesrvrate'] * df['dsthostcount']
    df['dsthost_diffsrv_count'] = df['dsthostdiffsrvrate'] * df['dsthostcount']
    df['dsthost_errors_srvcount'] = df['dsthostsrverrorrate'] * df['dsthostsrv
    df['dsthost_errors_srvcount'] = df['dsthostsrvrerrorrate'] * df['dsthostsrv
```

```

df['dsthost_samesrcport_srvcount'] = df['dsthostsamesrcportrate'] * df['dsth
df['dsthost_srvdiffhost_srvcount'] = df['dsthostsrvdiffhostrate'] * df['dsth
df = df.drop(columns=['numoutboundcmds'], errors='ignore')
df['srcbytes/sec'] = df.apply(lambda r: r['srcbytes'] / r['duration'] if r['
df['dstbytes/sec'] = df.apply(lambda r: r['dstbytes'] / r['duration'] if r['
df['suattempted'] = df['suattempted'].apply(lambda x: 0 if x == 0 else 1)
return df

def transform_preprocessing(X_raw, artifacts):
    X = X_raw.copy()
    ohe, service_encoding, scaler = artifacts["ohe"], artifacts["service_encodin
    ohe_arr = ohe.transform(X[["protocoltype", "flag"]])
    ohe_df = pd.DataFrame(ohe_arr, columns=ohe.get_feature_names_out(["protocolt
    X = pd.concat([X.drop(columns=["protocoltype", "flag"]), ohe_df], axis=1)
    max_rank = max(service_encoding.values())
    X["service"] = X["service"].map(service_encoding).fillna(max_rank + 1)
    X_scaled = pd.DataFrame(scaler.transform(X), columns=X.columns, index=X.inde
    return X_scaled[FINAL_FEATURES]

def unsupervised_infer(x_row, deploy_artifact):
    """Pure function of (x_row, deploy_artifact) -- deploy_artifact contains NO
    information (no dominant_label, no y_true_for_labeling); see notebook Sectio
    kind = deploy_artifact["type"]

    if kind in ("kmeans", "hierarchical", "dbscan"):
        dists = np.linalg.norm(deploy_artifact["centroids"] - x_row, axis=1)
        score = float(dists.min())

    elif kind in ("isolation_forest", "elliptic_envelope"):
        score = float(-deploy_artifact["estimator"].decision_function(x_row.res

    elif kind == "pca_reconstruction":
        pca = deploy_artifact["pca"]
        x_rec = pca.inverse_transform(pca.transform(x_row.reshape(1, -1)))[0]
        score = float(np.mean((x_row - x_rec) ** 2))

    else:
        raise ValueError(kind)

    return {"anomaly_score": score, "is_anomaly": bool(score > deploy_artifact["
        "threshold": deploy_artifact["threshold"]}]

@st.cache_resource
def load_artifacts():
    def _load(name):
        with open(os.path.join(ARTIFACT_DIR, name), "rb") as f:
            return pickle.load(f)
    return {
        "prep_binary": _load("prep_binary.pkl"),
        "prep_multi": _load("prep_multi.pkl"),
        "attack_category_encoding": _load("attack_category_encoding.pkl"),
        "best_binary_model": _load("best_binary_model.pkl"),
        "best_multi_model": _load("best_multi_model.pkl"),
        "unsup_deploy_artifact": _load("unsupervised_deploy_artifact.pkl"),
        "unsup_algo_name": _load("unsupervised_best_algorithm_name.pkl"),
    }

```

```

artifacts = load_artifacts()
decode_attack_category = {v: k for k, v in artifacts["attack_category_encoding"]}

st.title("Network Anomaly Detection")
st.caption("Binary (attack vs. normal), multi-class (attack category), and unsupervised (anomaly detection ({artifacts['unsup_algo_name']}}, label-free) for")

st.subheader("Connection features")
col1, col2, col3 = st.columns(3)
with col1:
    duration = st.number_input("duration", min_value=0.0, step=1.0)
    protocoltype = st.selectbox("protocoltype", ["tcp", "udp", "icmp"])
    service = st.text_input("service", value="http")
    flag = st.selectbox("flag", ["SF", "S0", "S1", "S2", "S3", "REJ", "RSTO", "RST", "SACK", "FIN", "ACK", "RST", "SACK", "FIN", "ACK"])
    srcbytes = st.number_input("srcbytes", min_value=0, step=1)
    dstbytes = st.number_input("dstbytes", min_value=0, step=1)
    land = st.selectbox("land", [0, 1])
    wrongfragment = st.number_input("wrongfragment", min_value=0, step=1)
    urgent = st.number_input("urgent", min_value=0, step=1)
    hot = st.number_input("hot", min_value=0, step=1)
    numfailedlogins = st.number_input("numfailedlogins", min_value=0, step=1)
    loggedin = st.selectbox("loggedin", [0, 1])
    numcompromised = st.number_input("numcompromised", min_value=0, step=1)
    rootshell = st.selectbox("rootshell", [0, 1])
with col2:
    suattempted = st.selectbox("suattempted", [0, 1, 2])
    numroot = st.number_input("numroot", min_value=0, step=1)
    numfilecreations = st.number_input("numfilecreations", min_value=0, step=1)
    numshells = st.number_input("numshells", min_value=0, step=1)
    numaccessfiles = st.number_input("numaccessfiles", min_value=0, step=1)
    numoutboundcmds = st.number_input("numoutboundcmds", min_value=0, step=1, value=0)
    ishostlogin = st.selectbox("ishostlogin", [0, 1])
    isguestlogin = st.selectbox("isguestlogin", [0, 1])
    count = st.number_input("count", min_value=0, step=1)
    srvcount = st.number_input("srvcount", min_value=0, step=1)
    serrorrate = st.number_input("serrorrate", min_value=0.0, max_value=1.0, step=0.01)
    srvserrorrate = st.number_input("srvserrorrate", min_value=0.0, max_value=1.0, step=0.01)
    rerrorrate = st.number_input("rerrorrate", min_value=0.0, max_value=1.0, step=0.01)
    srvrerrorrate = st.number_input("srvrerrorrate", min_value=0.0, max_value=1.0, step=0.01)
    samesrvrate = st.number_input("samesrvrate", min_value=0.0, max_value=1.0, step=0.01)
with col3:
    diffsrvrate = st.number_input("diffsrvrate", min_value=0.0, max_value=1.0, step=0.01)
    srvidffhostrate = st.number_input("srvidffhostrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostcount = st.number_input("dsthostcount", min_value=0, step=1)
    dsthostsrvcount = st.number_input("dsthostsrvcount", min_value=0, step=1)
    dsthostsamesrvrate = st.number_input("dsthostsamesrvrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostdiffsrvrate = st.number_input("dsthostdiffsrvrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostsamesrcportrate = st.number_input("dsthostsamesrcportrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostsrvdiffhostrate = st.number_input("dsthostsrvdiffhostrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthosterrorrate = st.number_input("dsthosterrorrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostsrverrorrate = st.number_input("dsthostsrverrorrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostrrerrorrate = st.number_input("dsthostrrerrorrate", min_value=0.0, max_value=1.0, step=0.01)
    dsthostsrvrrerrorrate = st.number_input("dsthostsrvrrerrorrate", min_value=0.0, max_value=1.0, step=0.01)

raw_input = dict(duration=duration, protocoltype=protocoltype, service=service, srcbytes=srcbytes, dstbytes=dstbytes, land=land, wrongfragment=wrongfragment, urgent=urgent, hot=hot, numfailedlogins=numfailedlogins, loggedin=loggedin, numcompromised=numcompromised, rootshell=rootshell, suattempted=suattempted, numroot=numroot, numfilecreations=numfilecreations, numshells=numshells, numaccessfiles=numaccessfiles, numoutboundcmds=numoutboundcmds, ishostlogin=ishostlogin, isguestlogin=isguestlogin, count=count, srvcount=srvcount, serrorrate=serrorrate, srvserrorrate=srvserrorrate, rerrorrate=rerrorrate, srvrerrorrate=srvrerrorrate, samesrvrate=samesrvrate, diffsrvrate=diffsrvrate, srvidffhostrate=srvidffhostrate, dsthostcount=dsthostcount, dsthostsrvcount=dsthostsrvcount, dsthostsamesrvrate=dsthostsamesrvrate, dsthostdiffsrvrate=dsthostdiffsrvrate, dsthostsamesrcportrate=dsthostsamesrcportrate, dsthostsrvdiffhostrate=dsthostsrvdiffhostrate, dsthosterrorrate=dsthosterrorrate, dsthostsrverrorrate=dsthostsrverrorrate, dsthostrrerrorrate=dsthostrrerrorrate, dsthostsrvrrerrorrate=dsthostsrvrrerrorrate)

```





model families with correctly fold-scoped SMOTE (Section 05) and separately over 6 unsupervised algorithm families with a proper train/validation/test split (Section 06). All three resulting models are deployed together via a **Flask REST API** and a **Streamlit** demo app (Section 07), so a caller gets both supervised predictions and an independent, label-free anomaly signal on every request.

**Final results, on held-out data the respective models never trained on:**

| Model                                              | Task                       | Test Accuracy | Precision | Recall | F1                       |
|----------------------------------------------------|----------------------------|---------------|-----------|--------|--------------------------|
| Gradient Boosting (imbalanced)                     | Binary (attack/normal)     | 99.91%        | 0.9996    | 0.9985 | <b>0.9991</b>            |
| LightGBM + SMOTE                                   | Multi-class (5 categories) | 99.90%        | 0.9990    | 0.9990 | <b>0.9990</b> (weighted) |
| Hierarchical Clustering (unsupervised, label-free) | Binary anomaly flag        | —             | 0.714     | 0.383  | <b>0.499</b>             |

The supervised models are near-perfect on 4 of 5 classes; the exception is **U2R** (precision 0.75 / recall 0.90 on only 10 test rows). The unsupervised model — evaluated honestly on a test partition it never touched during algorithm selection or threshold calibration — recovers real, meaningfully-above-random structure (PR-AUC = 0.668, ARI = 0.079, NMI = 0.067) with high precision but low recall: it is a conservative, high-confidence-when-it-fires detector, not a comprehensive one.

## Actionable insights

- count**, **srvcount**, and the **dsthost\_\*** error-rate/count features are the strongest supervised predictors and showed the cleanest visual separation as early as Section 03's EDA — network monitoring dashboards should prioritize these signals.
- icmp** traffic and non-**SF** connection flags are disproportionately associated with attacks (Section 03.2) and warrant tighter default firewall rules.
- U2R is the clear, specific weak point** of the supervised pipeline: only 42 training rows and 10 test rows exist for this class, and precision (0.75) lags recall (0.90) even after SMOTE — the model occasionally over-flags other classes as **U2R** rather than under-detecting it, likely a side effect of synthetic oversampling on such a tiny minority class.
- The unsupervised model is a high-precision, low-recall signal (0.714 / 0.383)** — it should be read as *"when this fires, take it seriously"* rather than *"if this doesn't fire, traffic is safe"*. Its real (ARI = 0.079, NMI = 0.067), if modest, alignment with true labels — found with zero label information used for fitting — is what gives it standalone value for catching attack patterns the supervised models were never trained to recognize.

## Recommendations

1. **Targeted U2R / R2L improvement:** cost-sensitive learning or a class-specific decision threshold, rather than relying solely on SMOTE, given how thin the U2R support is (42 train / 10 test rows) even after resampling.
2. **Widen the unsupervised search** if compute allows: a larger UNSUP\_SAMPLE\_SIZE (currently 6,000, split 3,000/1,500/1,500) would give Hierarchical/DBSCAN more data to find structure in, and more Optuna trials (currently 50) would explore the contamination/hyperparameter space more thoroughly — worth checking whether Isolation Forest or Elliptic Envelope close the recall gap with a larger budget.
3. **Combine the two signals operationally:** the /predict/all Flask endpoint already returns supervised + unsupervised predictions together. Given the unsupervised model's high-precision/low-recall profile, a sensible starting rule is: trust the supervised model's classification, but **escalate for human review whenever the unsupervised flag fires and disagrees with it** — since a low-recall-but-real signal firing on a case the supervised model called "normal" is exactly the profile of a potential novel/zero-day attack pattern.
4. **Periodic retraining:** It is a static, aging dataset; production traffic will drift. The unsupervised model in particular should be refit periodically on recent unlabeled traffic, since its entire value proposition is catching what supervised training never saw.

## Future scope

- Real-time streaming inference (Kafka/Flink) instead of batch/API-call scoring.
- A learned meta-model on top of [supervised probability, unsupervised anomaly score] , replacing the simple disagreement rule above once enough operational data exists to train one.
- Re-run the unsupervised study with a larger sample / more trials to check whether Hierarchical Clustering's win (PR-AUC 0.682 validation / 0.668 test) is stable, or whether Isolation Forest / Elliptic Envelope become competitive with more data — the current result is a single run, not a stability-tested conclusion.
- Containerize the Flask API (Docker) and place it behind a proper WSGI server (gunicorn) + load balancer for production traffic.

In [ ]: